Campus Shuttle Tracking & Demand Prediction

BCS 305 Software Architecture — Course Project Report

Canadian University Dubai · Fall 2026 · Instructor: Dr. Khaled Ammar

Team: **Abdullah** (GitHub `abdullahshammari796-ctrl`) · **Zuhair Halawa** (GitHub `zuhairhalawa`)

Student IDs: *OUTSTANDING — to be added by the team*

Version: draft of 2 November 2026

> **Status of this document.** This is a working draft. The architecture, models, prototype and test evidence it describes exist in the repository and were verified on 31 October – 1 November 2026. Items marked **OUTSTANDING** need information or actions that only the team or the instructor can provide (signatures, actual dates, personal reflection, rendered diagrams). Section 10 states how AI assistance was used; it must be completed truthfully by the team before submission.

1. Introduction and problem statement

The problem. Many UAE university campuses run shuttle buses between faculty buildings, student residences, car parks and the nearest metro station. Students cannot see where a shuttle is or when it will arrive, so they either wait at the stop — outdoors, in a climate where summer temperatures regularly exceed 40 °C — or miss the shuttle. The transport office, in turn, decides how many vehicles to run on which route from experience, because it has no record of where and when students actually wait.

Who has it and why it matters. *Students* lose time and comfort; students with reduced mobility are affected most. *Drivers* are interrupted by phone calls asking where they are. *Transport office staff* cannot see their fleet, cannot react when a vehicle stops reporting, and have no data to plan capacity. A system that shows live positions and honest arrival estimates, and that turns "I am waiting here" signals into demand history and forecasts, addresses all three.

Scope — included. Live shuttle tracking on a schematic campus map; arrival estimates per stop with an explicit confidence level; detection of delayed and lost GPS; trip management by drivers with a built-in GPS simulator; waiting requests; fleet and demand monitoring, route management and trip inspection by the transport office; historical demand trends and explainable demand forecasts with a model evaluation; role-based access, privacy controls and an audit log.

Scope — not included. Integration with a real university's single sign-on, student information system or vehicle telematics; native mobile apps and push notifications; timetable planning or automatic dispatching; payments; multiple campuses; Arabic localisation. All routes, stops, vehicles, accounts and historical demand in the prototype are **demonstration data**; nothing in this report describes the real operations of any university.

2. Stakeholders and requirements

2.1 Stakeholder analysis

Stakeholder	Interest / goal	Influence	What the system gives them	Main concern
Students (incl. students with reduced mobility)	Know when to go to the stop; spend less time waiting outdoors	Low individually, high collectively	Live map, ETAs with confidence, "I'm waiting here"	Accuracy; privacy (not being tracked)
Shuttle drivers	Do the job with minimal phone interaction; not be blamed for GPS faults	Medium	One-tap trip start/end, simulator for training, clear "what passengers see"	Distraction while driving; being monitored
Transport office staff (dispatchers)	See the fleet, react to incidents, plan capacity	High	Live operations view, remote trip termination, routes, trends, forecasts	Reliability of the data they act on
University IT / operations	Run the system securely and cheaply	High (approves deployment)	One process, one database file, no paid services	Maintainability, security, backups

Stakeholder	Interest / goal	Influence	What the system gives them	Main concern
Data-protection officer (UAE Federal Decree-Law 45/2021 on personal data)	Lawful, minimal processing of personal data	High (can block launch)	Data minimisation, pseudonymisation, retention limits, audit log	Location data of drivers; identities of students
University management (sponsor)	Better service at reasonable cost	High	Evidence (demand trends) for fleet decisions	Cost, adoption

2.2 Functional requirements

Status: ✅ implemented and verified · ◐ implemented with a stated limitation · ✗ not implemented.

ID	Requirement	Actor	Status
FR-01	Users sign in; every API call is authorised by role (student, driver, office)	All	✅
FR-02	Show active shuttles on a map of routes and stops, updated live	Student, Office	✅
FR-03	Show arrival estimates for a chosen stop, soonest first, with confidence	Student	✅
FR-04	Show route and stop information (routes serving a stop, description, waiting count)	Student	✅
FR-05	Register, replace and cancel a waiting request; see when it was served or expired	Student	✅
FR-06	Indicate delayed and lost GPS; never show an estimate without a position	Student, Office	✅
FR-07	Start a trip by choosing route and shuttle; end it (the trip ends on the device only after the server confirms)	Driver	✅
FR-08	Send location updates from device GPS or a simulator; buffer them when offline (bounded: see QA-4)	Driver	◐ device-GPS path implemented, not field-tested on a vehicle
FR-09	Show trip status, next stop and what passengers currently see	Driver	✅
FR-10	Report passenger load (seats available / filling up / full)	Driver	✅
FR-11	Monitor all active shuttles and their GPS state live	Office	✅
FR-12	See current waiting demand per stop live	Office	✅
FR-13	End a trip remotely with a mandatory, audited reason	Office	✅
FR-14	Manage routes: take in/out of service, edit attributes, create from stops	Office	◐ new routes use straight lines between stops
FR-15	Inspect trip history and the recorded GPS track of a trip	Office	✅
FR-16	View historical demand trends (per day, weekday × hour, per stop)	Office	✅ (synthetic + live data)

ID	Requirement	Actor	Status
FR-17	View explainable demand forecasts and compare models	Office	✅ (baseline models; see §6.7)
FR-18	Detect stale/lost GPS automatically; close trips silent for 30 minutes	System	✅
FR-19	Expire waiting requests after 20 minutes; mark them served when a shuttle arrives	System	✅
FR-20	Delete raw GPS after 30 days; erase waiting-request pseudonyms 24 h after closing	System	✅
FR-21	Record privileged office actions in an audit log	System	✅

2.3 Non-functional requirements as quality-attribute scenarios

Each scenario uses the six-part form (source, stimulus, artifact, environment, response, response measure). "Evidence" points to how it was verified; numbers are measured, not estimated. Full detail: `docs/architecture/quality-attribute-scenarios.md`.

ID	Quality	Scenario (source → stimulus → artifact, environment → response)	Response measure	Evidence (measured)
QA-1	Performance (freshness)	Driver app uploads a position → tracking + real-time gateway, normal operation → every connected viewer receives the new position and ETAs	p95 ≤ 1 s from upload to delivery	Local benchmark, 1,000 viewers: p95 **35–37 ms**, max **55–56 ms** (`npm run load-test`; same process, loopback, no browsers — see QA-1/QA-2 in `docs/architecture/quality-attribute-scenarios.md`)
QA-2	Scalability	3,000 students open the app at the start of term → one server → keeps delivering every update to everyone	max ≤ 1 s, no updates dropped for healthy clients	Local benchmark, 3,000 simulated viewers: max **108–113 ms**, 0 skipped. Not evidence for 3,000 real browsers on campus networks
QA-3	Availability of truthful information (GPS outage)	Shuttle GPS stops → tracking, trip active → viewers told "delayed", then "lost"; estimate hidden when lost; recovery on next position; trip auto-closed after 30 min	delayed within 22 s, lost within 62 s; 0 estimates shown in LOST; recovery on the first accepted fix	unit + integration tests; manual browser check
QA-4	Reliability (network loss)	Driver phone loses mobile data while GPS works, page stays open → driver app + tracking → positions are queued, delivered in order on reconnection; duplicates and out-of-order positions rejected	Bounded: no validated position lost for outages ≤ 10 min with the page open; beyond that, positions older than 10 min are rejected, the queue keeps the newest 500, and a page reload loses the queue	tests "shuffled offline backlog", "recovers when GPS resumes", uploader queue and end-trip tests; manual demo

ID	Quality	Scenario (source → stimulus → artifact, environment → response)	Response measure	Evidence (measured)
QA-5	Security (access control)	Unauthenticated caller or wrong role calls a protected endpoint → web API → request refused	100 % of 26 endpoints × 4 caller types answer 401/403; forged or expired tokens rejected; open live streams closed when the session expires; 5 failed logins → 5-min lockout	automated role-matrix test; token tests; live-stream expiry test
QA-6	Privacy	Any stored or displayed data → whole system → minimum personal data	students' location never collected; no driver identity in student-visible data; waiting pseudonym erased 24 h after closing; GPS traces deleted after 30 days	tests (view contents, retention job)
QA-7	Modifiability	Developer adds a new ETA algorithm or forecasting model → tracking / demand → added without touching services, API or UI	one new class + one line in the composition root	Strategy pattern (§7); model list in the UI is data-driven
QA-8	Testability	Developer runs the test suite → whole code base, developer PC, no network → all business rules exercised, time-dependent rules deterministic	full suite < 10 s	**117 tests in about 5 s** (`npm test`); 22 Chromium tests run separately
QA-9	Data consistency	Two drivers try to start trips on the same shuttle at the same moment → tracking + database → exactly one succeeds	100 %, guaranteed by the database	unique partial indexes + service check; tests

The **hard quality attribute** named in the proposal is *timeliness and honesty of shuttle information under unreliable GPS and mobile connectivity* (QA-1, QA-3, QA-4): the system must be fast when data is good, and must say so — instead of guessing — when it is not.

3. Planning and development process

3.1 Process followed and why

We used a **risk-driven incremental process**: a deliberately chosen architectural skeleton first, then increments that each pass through analysis, design, build and test — with the riskiest questions answered by prototypes before the features that depend on them, in the spirit of the spiral model (BCS 305 Module

02). This "middle path" — enough architecture up front to fix the quality attributes and the component boundaries, then incremental delivery inside that structure — suited this project because:

- the quality attributes were known early (§2.3), and the expensive-to-reverse decisions (process structure, real-time transport, where ETAs are computed, how outages are represented) had to be made before building screens;
- a two-person team cannot absorb the coordination cost of a heavyweight process, but the course has fixed gates (proposal, report, presentation) that reward documented decisions;
- the demand-prediction part follows the ML lifecycle in the instructor's recommended reading (RBC Borealis field guide): discovery and feasibility first, a baseline before any model, and an explicit decision about whether data justifies more (§6.7);
- the V-model idea from the module was applied to testing: each specification level has a matching test level (§3.4).

Risk (what we did not know)	How it was reduced	Result
Can one ordinary server push updates to hundreds or thousands of viewers fast enough?	Prototype of the SSE gateway + local load test	3,000 simulated viewers in one process, max 108–113 ms; a deployment test with real clients is still needed
How do we stay honest when GPS or the network fails?	State machine for signal quality; store-and-forward on the phone	Delayed → lost → recovered, tested
Can ETAs be computed robustly on routes that use the same road both ways?	Linear referencing with a search window	Tested; the load test found and we fixed a corner-snapping bug
Is there data to justify machine learning?	Feasibility check (Borealis lifecycle)	No real data → explainable baselines + synthetic data clearly labelled

3.2 Increments

1. *Skeleton*: component structure, composition root, persistence, authentication, event bus.
2. *Tracking*: trips, location ingestion, route snapping, ETA strategies, signal state machine, live feed.
3. *Student and driver interfaces*, GPS simulator, offline queue.
4. *Demand*: waiting requests, demand history, predictors, backtest, office analytics.
5. *Hardening*: role matrix, privacy retention, audit log, load test, architecture-conformance test, documentation.

3.3 Actual timeline

When	Activity	Evidence / status
Week 2–3 (18 Sep 2026: Module 02 session)	Team formation, topic choice	**OUTSTANDING — team to confirm dates**
Week 4	Proposal submitted / topic approved	**OUTSTANDING — team to confirm submission and approval date** (`docs/01-proposal.md` is the one-page proposal)
31 Oct – 1 Nov 2026	Increments 1–5 implemented, tested and documented in this repository with AI assistance (see §10)	Repository contents; `docs/evidence/`

When	Activity	Evidence / status
Weeks 10–13 (planned)	Team review of every artefact; diagrams rendered and checked; contributions in `docs/team/work-ownership-plan.md`	**Planned — to be updated with actual dates**
Week 12	Contribution concerns raised with the instructor if any	Requirement of the brief
Week 14 or 15	Presentation and live demonstration (`docs/presentation/`)	**OUTSTANDING**
Week 15	Final submission on Moodle	**OUTSTANDING**

3.4 Verification approach (V-model pairing)

Specification level	Matching test level	Where
Requirements (FR, QA scenarios)	Acceptance: scripted demo + browser walkthrough of UC1–UC5	`docs/demo-script.md` , `docs/test-plan.md`
System design (API, live feed)	System/integration tests over real HTTP and SSE	`test/integration/*.test.ts`
Architectural design (components, dependency rules)	Architecture-conformance test	`test/architecture.test.ts`
Module design (classes)	Unit tests	`test/unit/*.test.ts`

4. Analysis models

The full analysis models are in Appendix A: use case diagram (A.1), five fully dressed use case descriptions (A.2), activity diagram for *Run a trip* (A.3), system sequence diagram for *Run a trip and track it* (A.4), and state machines for the trip with its GPS signal (A.6) and for a waiting request (A.7). The primary use cases are:

- **UC1 Track shuttles and view arrival times** (Student)
- **UC2 Run a trip** — start, share location, end (Driver; GPS receiver as secondary actor)
- **UC3 Register a waiting request** (Student)
- **UC4 Monitor fleet and waiting demand**, extended by *End a trip remotely* (Office)
- **UC5 View demand forecast**, extended by *Explain a forecast* (Office)

The state machine for the trip is central to the design: an `Active` trip has the sub-states `AwaitingFix → Live ⇄ Stale → Lost`, driven by `after(20 s)` / `after(60 s)` time events and by accepted positions. The same four states exist as classes in the code (State pattern, §7.3), so the model and the implementation can be compared one-to-one.

5. Domain and structural models

Appendix A contains the domain model (A.8), two design class diagrams with attributes, operations, associations and multiplicities (A.9, A.10), the component diagram (A.11) and the deployment diagram (A.12).

Domain model in brief. A *Route* visits two or more *Stops* in order (association class *RouteStop*); a *Trip* runs one *Route* with one *Shuttle* and one *Driver* and records many *LocationFixes*; a *Student* places *WaitingRequests* at a *Stop*; *HourlyDemand* observations at each stop are the input of *DemandForecasts*. Invariants: a shuttle and a driver are each on at most one active trip; an ended trip never changes.

Components (each is a folder under `src/`):

Component	Responsibility	Provided interface(s)	Required interface(s)
Web API (`web/`)	HTTP adapter: routing, input parsing, authentication and role checks, error mapping, security headers	REST API	application services of all components
Real-time Gateway (`realtime/`)	Server-Sent Events: snapshot on connect, coalesced pushes, heartbeats, slow-client handling; `LiveView` combines tracking and demand data	Live feed	`TrackingService`, `WaitingService` (read only), event subscription
Identity & Access (`identity/`)	Password hashing, signed session tokens, login throttling, pseudonymisation	`AuthService`	`UserRepository`
Network Management (`network/`)	Routes, stops, shuttles; route geometry (`RoutePath`)	`NetworkReader`, `NetworkService`	route/stop/shuttle repositories, events
Tracking (`tracking/`)	Trips, position ingestion, route snapping, ETAs, GPS signal states	`TrackingService` (incl. vehicle views)	`NetworkReader`, trip/fix repositories, events
Demand & Forecasting (`demand/`)	Waiting requests, demand history, predictors, backtest	`WaitingService`, `DemandService`	`NetworkReader`, waiting/history repositories, events
Event Bus (`shared/`)	In-process publish/subscribe of domain events	`EventPublisher`, `subscribe`	—
Persistence (`persistence/`)	SQLite schema, migrations, repository implementations	all repository ports	SQLite file

Deployment. For the prototype everything runs as one Node.js process on one machine; browsers connect over HTTP on `localhost`. For production we recommend a campus virtual machine behind a TLS reverse proxy (HTTPS is required for phone GPS and for `Secure` cookies) and daily backup of the database file (Figure A.12).

6. Architectural analysis and trade-offs

6.1 Architectural drivers

From §2.3, in priority order: (1) timeliness and honesty of shuttle information under unreliable GPS/connectivity (QA-1, QA-3, QA-4); (2) many concurrent viewers at peak times (QA-2); (3) security and privacy of location and identity data (QA-5, QA-6); (4) modifiability and testability, because forecasting and ETA methods will change as data accumulates (QA-7, QA-8); and two constraints — a **two-person team** and **no paid services or cloud subscriptions** (the prototype must run on a laptop).

6.2 Candidate architectures

A. Layered modular monolith with an in-process event bus and server push (SSE). One deployable. Inside, components with explicit port interfaces (hexagonal style); components react to each other through domain events; browsers receive pushes over Server-Sent Events.

B. Microservices. Separate Tracking, Demand, Identity and Notification services behind an API gateway, connected by a message broker (e.g. RabbitMQ or Kafka), each with its own database; WebSockets for clients.

C. Simple monolith with client polling. One MVC application; browsers poll `/live` every few seconds.

D. Serverless backend-as-a-service. A hosted real-time database and cloud functions (Firebase-style); clients subscribe directly to the hosted database.

6.3 Comparison against the quality attributes

Scores 1 (poor) – 5 (excellent) were proposed in the AI-assisted draft and are justified in the right-hand column and, where possible, by measurements. Only option A's freshness is measured (local benchmark) and option C's polling load is arithmetic; every other score is judgement. **The team's own re-scoring is OUTSTANDING** (issue #2; evidence notes in `docs/adr/abdullah-technical-review.md`).

Driver (weight)	A Modular monolith + SSE	B Microservices	C Monolith + polling	D Serverless BaaS	Justification
Freshness & fan-out, QA-1/2 (3)	5	4	2	5	A: p95 35–37 ms at 1,000 viewers in a local benchmark. B adds broker and gateway hops. C: data is on average half a polling interval old (2.5 s at 5 s polling) and 1,000 viewers create 200 req/s even when nothing moves. D: native push.
Robustness to GPS/network loss, QA-3/4 (3)	4	4	3	4	All can implement store-and-forward on the phone. A keeps live state in memory and rebuilds it from the database after a restart (tested), but is a single process. B isolates failures but has more failure modes. C makes staleness detection a client concern.
Security & privacy, QA-5/6 (2)	4	3	4	2	A/C: one trust boundary, data stays on campus. B: more network surface and service-to-service authentication. D: driver locations and student activity stored by a third party outside campus control, which conflicts with data-minimisation and the brief's preference for no cloud subscriptions.
Modifiability & testability, QA-7/8 (2)	4	3	3	3	A: component import rules checked by a test, plus ports; the whole Node suite runs in-process in about 5 s. B: independent deployment, but cross-service changes and integration testing are costly. C tends towards a "big ball of mud". D: logic split between client and vendor functions, harder to test locally.

Driver (weight)	A Modular monolith + SSE	B Microservices	C Monolith + polling	D Serverless BaaS	Justification
Operational simplicity for a team of two (3)	5	1	5	3	A/C: `npm install && npm start`, one database file. B: ≥ 5 deployables plus broker and gateway. D: no server, but vendor accounts, quotas and lock-in; offline demonstration is hard.
Growth beyond one campus (1)	3	5	2	5	A scales vertically; the `EventPublisher` port is the seam for a shared broker if several instances are ever needed.
Weighted total (max 70)	**61**	**44**	**46**	**51**	

6.4 Decision

We chose **A — a layered, modular monolith with an in-process event bus and Server-Sent Events**, the simplest architecture that meets every driver. It gives the best measured freshness, keeps all personal data in one place under one access-control layer, is operable by a two-person team, and keeps the evolution path open: each component already depends only on interfaces, so extracting the real-time gateway or replacing the event bus with a broker would not change business code. Microservices would buy independent scaling that one campus does not need, at a cost a two-person team cannot pay. The supporting decisions are recorded as architecture decision records:

ADR	Decision	Main alternative rejected
0001	Modular monolith with ports and adapters	Microservices; plain MVC
0002	Server-Sent Events for live updates; positions uploaded by REST	WebSockets; polling
0003	SQLite through Node's built-in `node:sqlite`, WAL mode	PostgreSQL server; in-memory only
0004	ETAs computed on the server; GPS freshness as an explicit state machine	ETAs in each browser
0005	Explainable seasonal baselines for demand; no trained ML model yet	Gradient boosting / neural model on synthetic data
0006	HttpOnly signed session cookies with role checks at the edge	JWT in localStorage
0007	Offline schematic SVG map instead of map tiles	Third-party map tiles
0008	TypeScript run directly by Node 24 (type stripping), no build step	Bundler + framework

6.5 Trade-offs, sensitivity points and risks

- **Single process (sensitivity point for availability).** If the process stops, live information stops. Mitigations: all trips and positions are persisted before memory is updated; on restart live state is rebuilt from the database (`TrackingService.recover`, tested); browsers reconnect automatically and receive a full snapshot. For a campus information service, a short outage during which students fall back to the printed timetable is acceptable; a lost record is not.
- **Synchronous SQLite calls (sensitivity point for latency).** Each statement blocks the event loop while it runs; at this data size that is short, but it depends on the query, the data volume and the disk. Fan-out was unaffected at 3,000 simulated viewers in the local benchmark; the risk returns if the database sits on a slow network drive.

- **In-process event bus (trade-off: simplicity vs horizontal scaling).** Fine for one instance. A second instance would need a shared broker behind the existing `EventPublisher` interface (ADR-0002).
- **SSE behind proxies.** Some proxies buffer streaming responses; the server sends `X-Accel-Buffering: no` and heartbeats every 15 s, and the client reconnects after 40 s of silence.
- **Device clocks.** Phones may have wrong clocks; every upload carries `sentAt`, and the server shifts all positions by the measured offset (tested with a 5-minute skew).

6.6 How the architecture addresses the cross-cutting concerns

Real-time communication and concurrent viewers. Drivers upload by ordinary REST; viewers receive Server-Sent Events. Each message is serialised once and written to every connection; all positions of one upload batch are coalesced into a single message per trip; a slow client whose socket buffer exceeds 256 KB skips incremental updates and receives a fresh snapshot when it drains, so one slow phone cannot grow server memory. On every (re)connection a snapshot replaces client state, so a client can never be left with a partial picture. A student page re-reads its own waiting request only after a snapshot, an arrival at its stop or a *fall* in that stop's waiting count, so students joining a busy stop do not make everyone else's page send requests.

ETA calculation. The route is *linear-referenced*: each GPS position is projected onto the route polyline to obtain "metres along the route". A search window around the previous progress keeps the shuttle on the correct lane of roads used in both directions; small backward jitter is ignored. ETA = remaining distance ÷ speed + dwell time at each intermediate stop. The default `BlendedSpeedEta` uses 60 % of the observed moving speed (exponentially smoothed, standing still ignored) and 40 % of the route's planned speed, clamped to 8–50 km/h, so one traffic jam or one GPS spike cannot produce absurd estimates. Estimates carry an absolute arrival time; browsers count down using a server-time offset, so device clocks do not distort them.

GPS outages and stale data. The `SignalState` machine (§7.3) classifies every active trip as awaiting fix, live, stale (> 20 s) or lost (> 60 s), measured from the newest accepted position in server-corrected time. Live estimates are shown as is; stale ones are labelled approximate and become "Due" when their time has passed; lost trips show **no** estimate, only "last seen". A delivered backlog whose newest position is already old is classified as stale immediately. Trips silent for 30 minutes are closed automatically, so no "ghost" shuttles remain. On the driver side, positions recorded during a mobile-network outage are queued in the open page and delivered on reconnection, within the bounds stated in QA-4 (newest 500 kept, at most 10 minutes old on arrival, lost if the page is closed). Ending a trip takes effect on the device only after the server confirms it. After a page reload, the driver app resumes the location source of the trip's latest accepted position; if none has been accepted yet, it shares nothing until the driver chooses, so a device-GPS trip never silently switches to simulated positions.

Persistence and data consistency. SQLite with write-ahead logging and foreign keys. Positions are written in one transaction per batch *before* in-memory state changes. Unique partial indexes (`… WHERE status = 'ACTIVE'`) make it impossible — even under concurrent requests — for a shuttle or a driver to have two active trips, and a student to have two open waiting requests. Positions are de-duplicated and ordered per trip; ended trips reject further positions (tested).

Security and role-based access. Passwords are salted scrypt hashes. Sessions are HMAC-signed tokens in `HttpOnly`, `SameSite=Strict` cookies (8-hour expiry), checked on every request together with the account's existence. A live SSE stream is authorised when it opens and its session's expiry is re-checked before every push and at every 15 s heartbeat; an expired stream receives `session-expired` and is closed. Tokens are stateless, so sessions **expire** but cannot be **revoked** early: logout deletes the cookie in that browser only,

and there is no server-side revocation list (a known prototype limitation). Roles are enforced at the edge by route prefix (`/api/student`, `/api/driver`, `/api/office`), and ownership inside services (a driver can only post to their own trip). State-changing calls require a custom header (CSRF defence in depth). A strict Content-Security-Policy with no inline scripts or styles, `X-Frame-Options: DENY`, `nosniff`, no-referrer and `no-store` on API responses are sent; all user-visible text is HTML-escaped. Five failed logins lock a username for five minutes. Office actions are audited.

Privacy. Data minimisation by design: students never send their location (they choose a stop); student-visible data never contains driver identity; waiting requests are linked to students only through a keyed pseudonym that is erased 24 h after the request closes; drivers' positions are collected only during an active trip and raw traces are deleted after 30 days; the map uses no third-party tiles, so viewers' locations are not disclosed to anyone. Details and a data inventory: `docs/privacy-and-security.md`.

Performance and scalability. A local benchmark on a Windows 11 desktop PC (Intel Core i7-14700F, 16 GB RAM, Node 24.14.1) gave: 1,000 simulated viewers p95 35–37 ms, max 55–56 ms; 3,000 viewers p95 90–98 ms, max 108–113 ms; process memory 311–332 MB including the 3,000 test clients. The viewers ran in the same process as the server over loopback, shared one demo session, and rendered nothing, so these numbers describe the server's fan-out, not the experience of 3,000 real students on campus Wi-Fi (method and remaining uncertainties: `docs/architecture/quality-attribute-scenarios.md`). The scaling path is vertical first, then a broker behind `EventPublisher` and several instances behind a load balancer.

Maintainability and testability. Constructor injection everywhere; the composition root (`src/container.ts`) is the only place that knows concrete classes. A `Clock` port makes every time rule deterministic in tests. `test/architecture.test.ts` enforces the component diagram's dependency rules on the code's imports: tracking and demand never import each other; business components see the network component only through types; only the composition root and seeding know persistence classes. It checks imports only, not the operations, associations or multiplicities in the class diagrams; those were compared with the code by hand (1 November 2026).

6.7 Demand prediction and the ML development lifecycle

The instructor's recommended reading (RBC Borealis, *Field Guide to Machine Learning Product Development*) describes a lifecycle of **discovery → feasibility → design requirements → model and product development → production → support and maintenance**, and stresses starting from a baseline and checking whether ML is needed at all. We applied it as follows:

Lifecycle stage	What we did
Discovery	Problem framed as "how many students will wait at each stop in each hour", to help the office allocate shuttles. Data sources identified: waiting requests and trips recorded by this system.
Feasibility	No real historical data exists for any campus. A trained model would have nothing real to learn from, and its accuracy could not be meaningfully evaluated. Decision: build **explainable baselines** and the full pipeline; use **synthetic data, clearly labelled**, only to demonstrate and test it.
Design requirements	Office staff must be able to see *why* a number is shown and how uncertain it is: every forecast lists its past observations, weights and a low–high range; "no history" is shown instead of a guess.
Model & product development	Four strategies behind one interface: *seasonal average* (same weekday and hour, 4 weeks), *weighted seasonal* (6 weeks, recent weeks weigh more), a robust *same-weekday median* (6 weeks) and a naive *hour-of-day mean* that a useful model must beat. A rolling-origin backtest computes MAE, RMSE, bias and range coverage without look-ahead (tested).
Production	Forecasts are computed on request from the live database; every live waiting request automatically becomes training/evaluation data.

Lifecycle stage	What we did
Support & maintenance	The backtest table is the monitoring tool: when enough real data exists (we suggest at least one full semester), rerun the evaluation on live data only; introduce a trained model only if it beats the seasonal baseline on held-out real weeks.

Backtest on the deterministic synthetic history (14 days, 1,890 stop-hours): seasonal average MAE 1.45, weighted seasonal 1.44, same-weekday median 1.42, hour-of-day mean 1.98 requests per stop-hour. The full `npm run evaluate` command was reproduced on Node 24.22.1 in ChatGPT Work on 1 November 2026, and again on Windows 11 with Node 24.14.1 during the owner review. **These numbers only show that the pipeline works and that weekday seasonality matters in data generated with weekday seasonality; they are not evidence of real-world accuracy.** Full description, assumptions of the generator and limitations: `docs/demand-prediction.md`.

7. Design patterns applied

Five patterns are applied; each has a class diagram in Appendix A (A.13–A.17) and is visible in the named source files.

7.1 Strategy — ETA estimation and demand forecasting (Figure A.13)

Problem. ETA and forecasting methods must change as we learn (e.g. a trained model later) and must be compared side by side, without editing the services that use them. *Solution.* `EtaStrategy` (`NominalSpeedEta`, `BlendedSpeedEta`) is injected into `TrackingService`; `DemandPredictor` (`SeasonalAveragePredictor`, `WeightedSeasonalPredictor`, `SameWeekdayMedianPredictor`, `HourOfDayMeanPredictor`) implementations are injected into `DemandService`, which also backtests all of them. *Consequence.* QA-7 is met: a new algorithm is a new class plus one line in `src/container.ts`; the office UI lists models from the API. *Code:* `src/tracking/eta/`, `src/demand/prediction/`.

7.2 Observer (publish/subscribe) — domain events (Figure A.14)

Problem. A shuttle update must reach many independent receivers — hundreds of browsers and the waiting-request module — without the tracking module knowing any of them; tracking and demand must stay decoupled. *Solution.* `EventBus<DomainEvent>` with typed subscriptions. Observers are **callbacks**, not classes implementing an interface: a handler has the function type `EventHandler<E> = (event: E) => void`; `SseHub` registers arrow functions in its constructor and the composition root registers a lambda that calls `WaitingService.markServed` on `stop.arrived`. Publishers depend only on the `EventPublisher` interface. Each handler runs inside `try/catch`, so one failure cannot break the publisher. *Consequence.* New reactions (e.g. notifications) need no change in tracking; the bus is the seam for a future broker. *Code:* `src/shared/event-bus.ts`, `src/realtime/sse-hub.ts`, `src/container.ts`.

7.3 State — GPS signal of an active trip (Figure A.15)

Problem. What passengers may be told depends on the freshness of GPS data, and the rules for reacting to new positions and to time passing differ per situation. *Solution.* `SignalState` with `AwaitingFixState`, `LiveState`, `StaleState`, `LostState`; the `TripTracker` context delegates `onFix`, `onTick` and `present(etas)` to its current state. States are stateless singletons. *Consequence.* No status `switch` statements spread across the code; the UML state machine (A.6) maps one-to-one onto classes. *Code:* `src/tracking/signal-state.ts`, `src/tracking/trip-tracker.ts`.

7.4 Repository (ports and adapters) — persistence (Figure A.16)

Problem. Business rules must not depend on SQL and must be testable without a database. *Solution.* Each component declares repository interfaces (`TripRepository`, `LocationFixRepository`, `WaitingRequestRepository`, …); `src/persistence/` implements them for SQLite, and in-memory versions exist for tests. *Consequence.* `TrackingService` is unit-tested with in-memory repositories; the database could be replaced by PostgreSQL by writing new adapters only. *Code:* `src/*/ports.ts`, `src/persistence/`.

7.5 Adapter — interchangeable GPS sources (Figure A.17)

Problem. The system must be demonstrable without vehicles, yet the demonstration must exercise exactly the same upload path as a real phone. *Solution.* `DeviceGps` adapts the browser Geolocation API to a small `start(onFix, onError) / stop()` contract; `SimulatedGps` implements the same contract by generating positions from route geometry. Because the driver app is plain JavaScript, `GpsSource` is a **conceptual (duck-typed) contract** documented in `gps.js`, not a declared interface; Figure A.17 labels it so. The trip-running code uses only `start`/`stop`; the demo control panel also calls simulator-only operations (speed, outage, glitch). Positions flow from the `onFix` callback into the `LocationUploader`. *Consequence.* The server processes both sources identically. It stores each position's `source` only for the record and to tell a reloaded driver page which source to resume, so a device-GPS trip never continues with simulated positions. *Code:* `public/js/gps.js`.

Supporting design techniques: *Dependency Injection* with a *Composition Root* (`src/container.ts`), *Facade*-like application services per component, and *Linear Referencing* for route geometry.

8. Prototype

8.1 What was built

A complete, locally runnable web application (`npm install`, `npm start`, open `http://localhost:3000`):

- **Student**: live map with route filter; stop selection (remembered per device); next arrivals with minutes, clock time, distance, load and signal badge; delayed/lost indicators; "I'm waiting here" with route option, cancel, served/expired feedback; connection banner.
- **Driver**: start trip (route, shuttle, GPS source); live trip card with next stop and ETA, GPS and upload status, "what passengers see"; passenger load; end trip with confirmation; resume after reload with the same location source; GPS simulator with speed control, GPS-outage, network-loss and glitch-injection buttons; activity log.
- **Transport office**: KPI tiles, fleet map with waiting badges, waiting per stop, active-trip table with remote termination; demand forecast (model and date), hourly chart, stop × hour heat map with per-cell explanation, historical trends, model backtest; route management (service status, attributes, new routes); trip history with GPS track and gap statistics; system health, privacy/retention summary and audit log.
- **Tooling**: deterministic demo seed, headless fleet simulator (`npm run simulate`), load test, demand evaluation, 117 Node tests (unit, integration, an architecture-conformance test and UML-export validation), 22 Chromium browser tests (roles, use cases, accessibility), and a GitHub Actions workflow that runs them on every pull request to `main`.

The two primary end-to-end use cases demonstrated are UC2 *Run a trip* with UC1 *Track shuttles* (driver → server → many students), and UC3 *Waiting request* feeding UC4/UC5 (student → office demand view → forecast). Both are covered by automated integration and Chromium browser tests and were walked

through in a browser on 31 October – 1 November 2026.

8.2 What was not built

Single sign-on and integration with university systems; native apps and push notifications; HTTPS configuration (development runs on `localhost`); field test with a real vehicle and phone GPS; timetable data (no schedule fallback when GPS is lost — the system shows "last seen" instead); multi-instance deployment; Arabic localisation; formal accessibility audit (automated axe WCAG checks and keyboard tests pass in Chromium, but no screen-reader session or physical-device check was done); tests in browsers other than Chromium; a trained ML model (deliberately, §6.7).

8.3 How the prototype maps onto the architecture

Component (Figure A.11)	Folder	Key classes	Tests
Web API	`src/web/`	`createApp`, route modules, `middleware.ts`	`test/integration/api.test.ts`
Real-time Gateway	`src/realtime/`	`SseHub`, `LiveView`	`test/integration/realtime.test.ts`
Identity & Access	`src/identity/`	`AuthService`, `TokenService`, password hasher	`test/unit/security-and-events.test.ts`
Network Management	`src/network/`	`NetworkService`, `RoutePath`	`test/unit/route-path.test.ts`
Tracking	`src/tracking/`	`TrackingService`, `TripTracker`, `SignalState`, ETA strategies	`test/unit/tracking-service.test.ts`, `trip-tracker`, `signal-state`, `eta`
Demand & Forecasting	`src/demand/`	`WaitingService`, `DemandService`, predictors, `backtest`	`test/unit/prediction.test.ts`, API tests
Event Bus	`src/shared/`	`EventBus`	unit tests
Persistence	`src/persistence/`	SQLite repositories, in-memory repositories, migrations	API and recovery tests
Composition root	`src/container.ts`	`buildContainer`	all integration tests
Browser clients	`public/`	`student.js`, `driver.js`, `office.js`, `gps.js`, `map.js`, `live-feed.js`	`e2e/*.spec.ts` (Chromium), `test/unit/simulated-gps.test.ts`, `driver-trip-ending.test.ts`

8.4 Technology choice

We chose **TypeScript on Node.js 24 LTS with Express 5 and SQLite (Node's built-in `node:sqlite`)**, and plain HTML/CSS/JavaScript modules in the browser. Node's event loop handles thousands of long-lived SSE connections cheaply, which is exactly the real-time workload; TypeScript makes the component interfaces (ports) explicit and checked; Node 24 runs TypeScript directly, so there is no build step to break; `node:sqlite` gives a transactional database with zero installation; and the whole stack is free, widely

documented and runs offline on any student laptop. The runtime has a single third-party dependency (Express). The trade-offs: `node:sqlite` is still marked experimental in Node 24 (we pin `engines` and test on 24.14), and without a front-end framework the UI code is more manual.

8.5 Evidence

Check	Result (31 Oct – 1 Nov 2026, Windows 11, Node 24.14.1)
`npm run typecheck`	passes, no errors
`npm test`	117 tests, 117 passed, 0 failed — `docs/evidence/test-run.txt`
`npm run test:browser`	22 Chromium tests, 22 passed (Playwright 1.64.1, Chrome for Testing 156) — `docs/evidence/browser-test-run.txt`
`npm audit`	0 vulnerabilities in all dependencies, including development tools — `docs/evidence/npm-audit.txt`
GitHub Actions (`verify`)	passed on `main` after PRs #12 and #13 (run 41341908650); required for merging into `main`
`npm run load-test` (1,000 viewers; local benchmark)	31 Oct: p50 23 ms, p95 35 ms, p99 47 ms, max 56 ms. 1 Nov: p50 25 ms, p95 37 ms, p99 48 ms, max 55 ms; 60,000/60,000 delivered; 0 skipped — `docs/evidence/load-test-1000*.txt`
`npm run load-test -- --viewers 3000 --rounds 10`	31 Oct: p95 90 ms, max 108 ms. 1 Nov: p95 98 ms, max 113 ms; 90,000/90,000 delivered; 0 skipped — `docs/evidence/load-test-3000*.txt`. Same process, loopback, one shared session, no browsers
`npm run evaluate`	see §6.7 — `docs/evidence/demand-evaluation.txt`
Browser walkthrough (student, driver, office)	all three interfaces exercised against a running server with the fleet simulator (`docs/test-plan.md` §5), and automated in Chromium since PR #13
`npm run uml` / `npm run uml:check`	2 Nov: all 16 diagrams rendered with the checksum-verified PlantUML MIT 1.2026.9 (Java 21, Smetana) and validated (complete, unclipped, from the current sources). An independent render of the same sources, made with ChatGPT Work at Zuhair's request (Java 17, Linux), matched in structure. Appendix A prints every diagram at 62 % or more of its rendered size; wide ones get overview and detail pages. The team's own visual check is still open (issue #1)

9. Reflection

> **OUTSTANDING — to be written by the team.** The brief rewards honest reflection. It must be your own: what you would do differently, what you learned, and how the work was shared. Useful prompts: Which decision would you revisit with real data (e.g. the 20 s / 60 s thresholds, the 60/40 speed blend)? What did using an AI assistant make easier, and what did you have to check or change yourselves? What would you test on a real shuttle first?

Factual engineering observations recorded while building the prototype, which the team may draw on:

- **Measuring found a real defect.** The first load-test run hung on one update. The cause was a route-snapping tie-break that preferred the earlier of two segments meeting at a corner, placing the shuttle up to a few metres behind its true position. A regression test now covers it. A test suite that only checked behaviour away from corners would not have found this.

- **Shutdown order matters.** An integration test revealed that the real-time gateway could still run deferred work after the database had been closed; the gateway now cancels pending work on close.
- **An independent review found three real defects** (1 November 2026): the driver app abandoned a trip locally even when the server had not confirmed the end; open live streams outlived their session; and the network-loss guarantee and load-test results were stated more strongly than the evidence supported. All were reproduced before fixing, and regression tests now cover the two code defects.
- **Browser tests found UI defects that unit tests could not** (PR #13): a waiting request that stayed on screen after it expired or was served while offline, keyboard focus lost on every countdown refresh, and insufficient contrast. The owner review of PRs #12/#13 then found two more: reloading a device-GPS trip switched it to simulated positions, and every student waiting at a stop re-fetched their request whenever anyone joined. Each was reproduced by a failing test before the fix.
- **Simulation is not reality.** All GPS behaviour was exercised with the simulator; real phones produce noisier positions, longer gaps and battery-saving pauses. The thresholds are therefore configurable, and a field test is the first thing we would do next.
- **No data, no model.** The most important ML decision was *not* to train a model on invented data.

10. Contribution statement and AI-use statement

10.1 Contribution statement

> **OUTSTANDING — must be completed and signed by every member.** List only work each member actually did. The suggested division of remaining work is in `docs/team/work-ownership-plan.md`; record what actually happened, not the plan. The instructor may ask any member to explain any diagram or decision.

Member	Contributions (design, models, code, tests, documentation, presentation)	Signature	Date
Abdullah	*to be completed*		
Zuhair Halawa	*to be completed*		

10.2 AI-use statement

> **Draft — the factual part below was recorded by the AI assistant; the team must review it, add their own use of AI tools if any, and complete "What we changed".**

- **Tools**: Claude Code (Anthropic) desktop application, model Claude Opus 5.5, used from Abdullah's computer and GitHub account; ChatGPT (OpenAI), including its Work mode, used from Zuhair's GitHub account (PRs #12 and #13, the review of PR #14 and the first UML render). The details are in `docs/team/ai-use-statement.md`.
- **When**: 31 October – 1 November 2026.
- **What it was used for**: reading the official project description and the Module 02 slides; analysing architecture alternatives; generating the application code, automated tests, load test and demo scripts; writing the PlantUML diagram sources; drafting this report, the proposal, ADRs, test plan and presentation materials; running the type checker, tests and load tests and fixing the defects they revealed.

- **What it was not used for**: it did not create the team, choose the topic for the team, submit anything for marking. Repository setup and later AI-assisted commits were explicitly requested by the respective account owners; see the current AI-use statement. It did not invent rubric details, signatures, timelines, contributions or real-world data; those are marked OUTSTANDING.
- **What we changed / verified ourselves**: **OUTSTANDING — the team must list the reviews, corrections and additions they made.**

Appendix A (UML diagram set) follows. Appendix B — requirements traceability: `docs/00-requirements-traceability.md`. *Appendix C — test plan and results:* `docs/test-plan.md`.

Appendix A — UML diagram set

Campus Shuttle Tracking & Demand Prediction · BCS 305 Software Architecture · Fall 2026

All diagrams are drawn with **PlantUML** (a text-based UML tool) using standard UML 2 notation. The editable sources are in `docs/uml/*.puml`; rendered SVG/PNG files go to `docs/uml/out/` (`npm run uml`). The class, component and sequence diagrams were written from the actual source code and were compared with it member by member on 1 November 2026 (corrections are listed in `docs/test-plan.md` §4). They were compared again in the owner review after PRs #12 and #13, which corrected diagrams 02, 04, 05, 07–12, 14 and 16 (`docs/test-plan.md` §4, item 13). **Rendering (2 November 2026):** all 16 sources render with the checksum-verified PlantUML MIT 1.2026.9 (built-in Smetana layout, no Graphviz). `npm run uml` validates every export (complete, unclipped, rendered from the current source) and records it in `docs/uml/out/manifest.json`; `npm run uml:check` repeats that check without Java. In this PDF a diagram is never printed below 60 % of its rendered size: one that is too large for a page at that scale gets a vector overview page (zoom in on screen) followed by overlapping detail pages. Only one aspect is checked automatically: `test/architecture.test.ts` fails if a source file imports a component that the component diagram does not allow. It does **not** check UML operations, attributes, associations, multiplicities or sequence messages; those were reviewed by hand and can drift when the code changes. Where a diagram writes a type as `{…}`, the code uses an inline TypeScript object type with no class name.

#	Diagram	Required by the brief	Source file
A.1	Use case diagram	Yes	`01-use-case-diagram.puml`
A.2	Use case descriptions (5 primary use cases)	Yes (≥ 3)	this document
A.3	Activity diagram — Run a trip	Yes	`02-activity-run-trip.puml`
A.4	System sequence diagram — Run a trip and track it	Yes	`03-system-sequence-diagram.puml`
A.5	Design sequence diagram — processing a location upload	Extra	`04-sequence-location-update.puml`
A.6	State machine — Trip and GPS signal	"Where relevant"	`05-state-machine-trip.puml`
A.7	State machine — Waiting request	"Where relevant"	`06-state-machine-waiting-request.puml`
A.8	Domain model (conceptual class diagram)	Yes	`07-domain-model.puml`
A.9	Design class diagram 1/2 — tracking, network, real-time	Yes	`08-class-diagram-tracking.puml`
A.10	Design class diagram 2/2 — demand, forecasting, identity	Yes	`09-class-diagram-demand-identity.puml`
A.11	Component diagram	Yes	`10-component-diagram.puml`
A.12	Deployment diagram	Yes	`11-deployment-diagram.puml`
A.13–A.17	Design pattern class diagrams (Strategy, Observer, State, Repository, Adapter)	Yes (≥ 3 patterns)	`12–16-pattern-*.puml`

A.1 Use case diagram

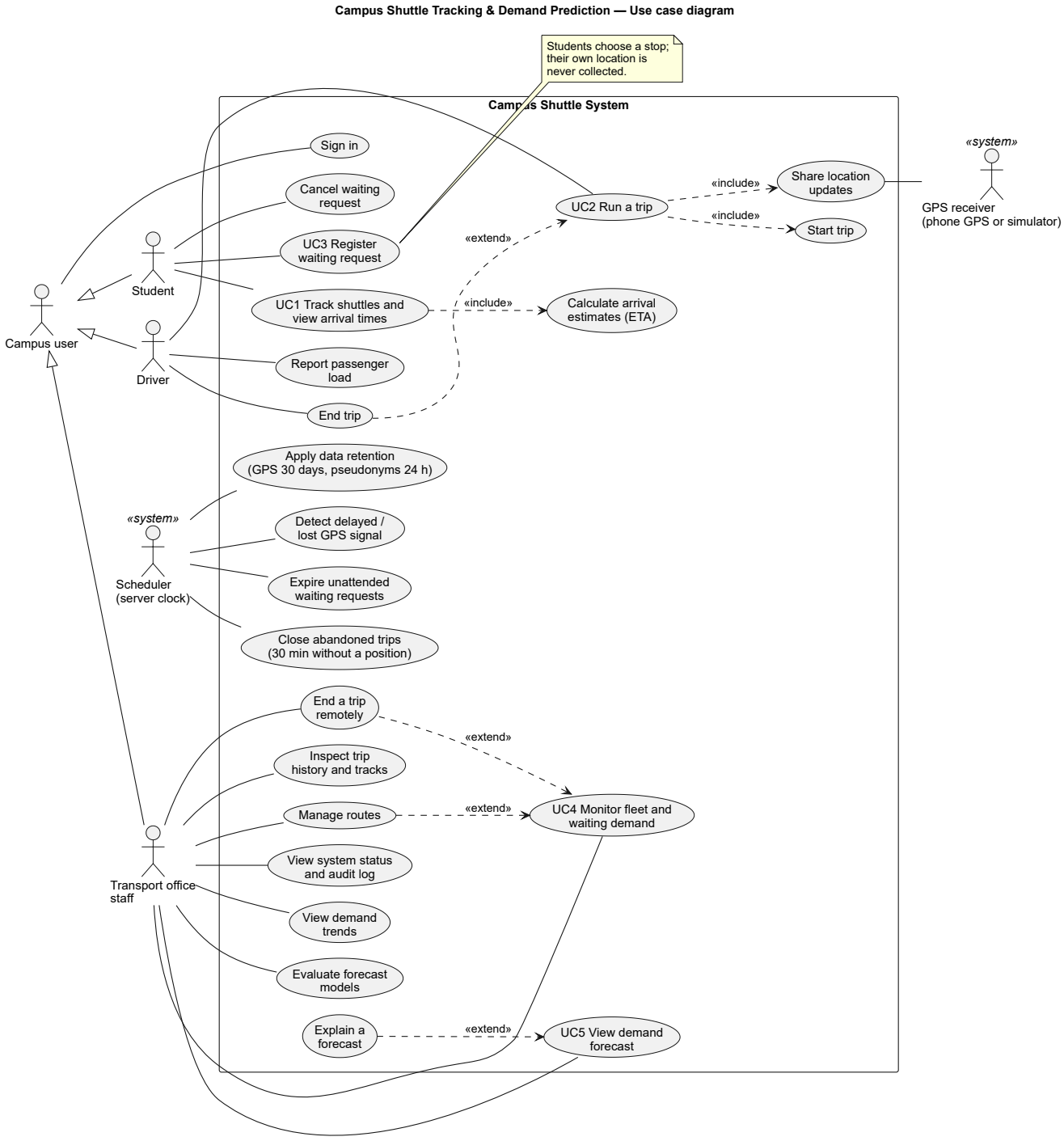


Figure A.1 — Use case diagram — overview at 55 % (read the 2 detail pages that follow, at 83 %; on screen, zoom in)

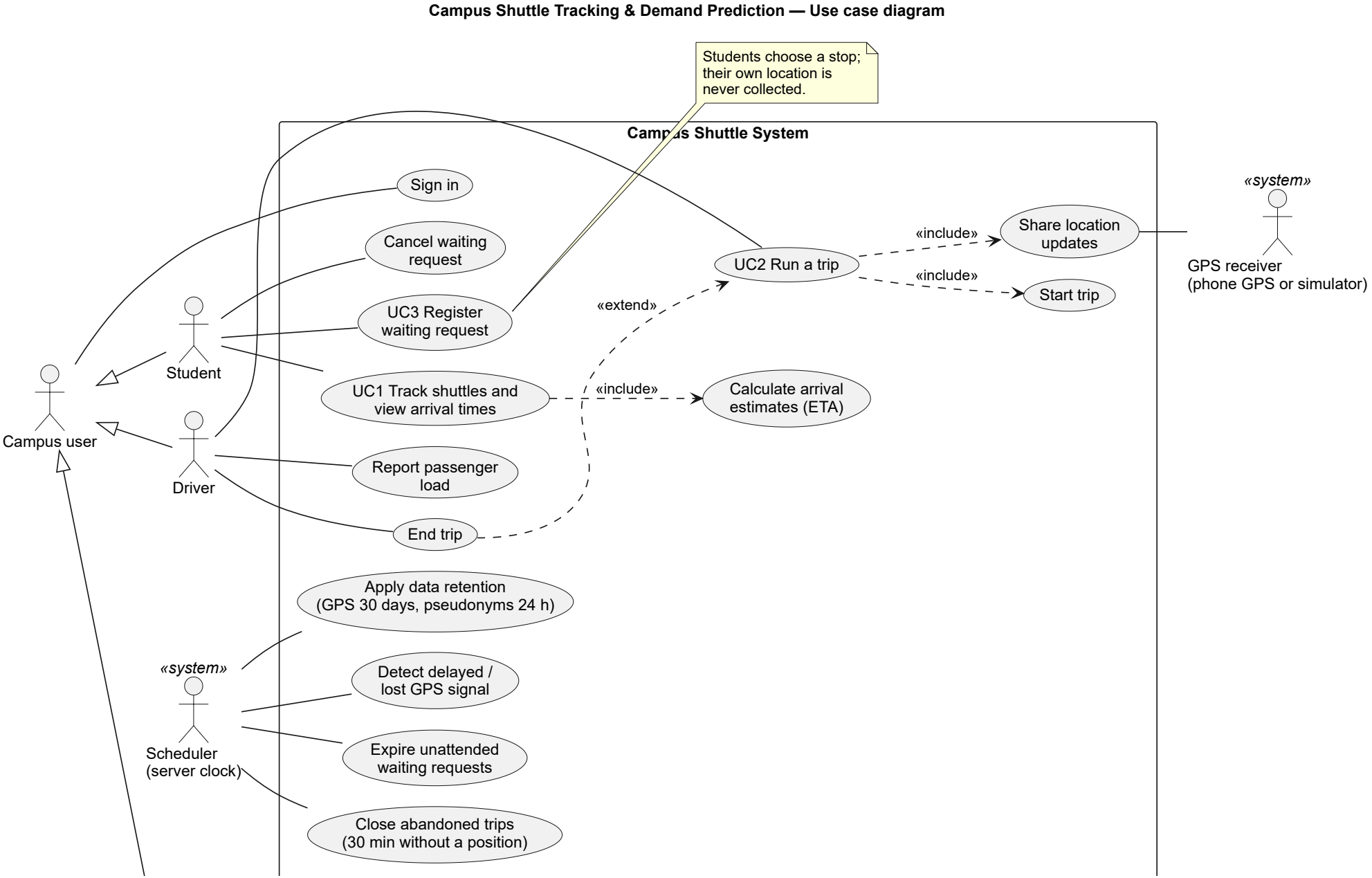


Figure A.1 — Use case diagram — detail 1 of 2 (row 1 of 2), at 83 %

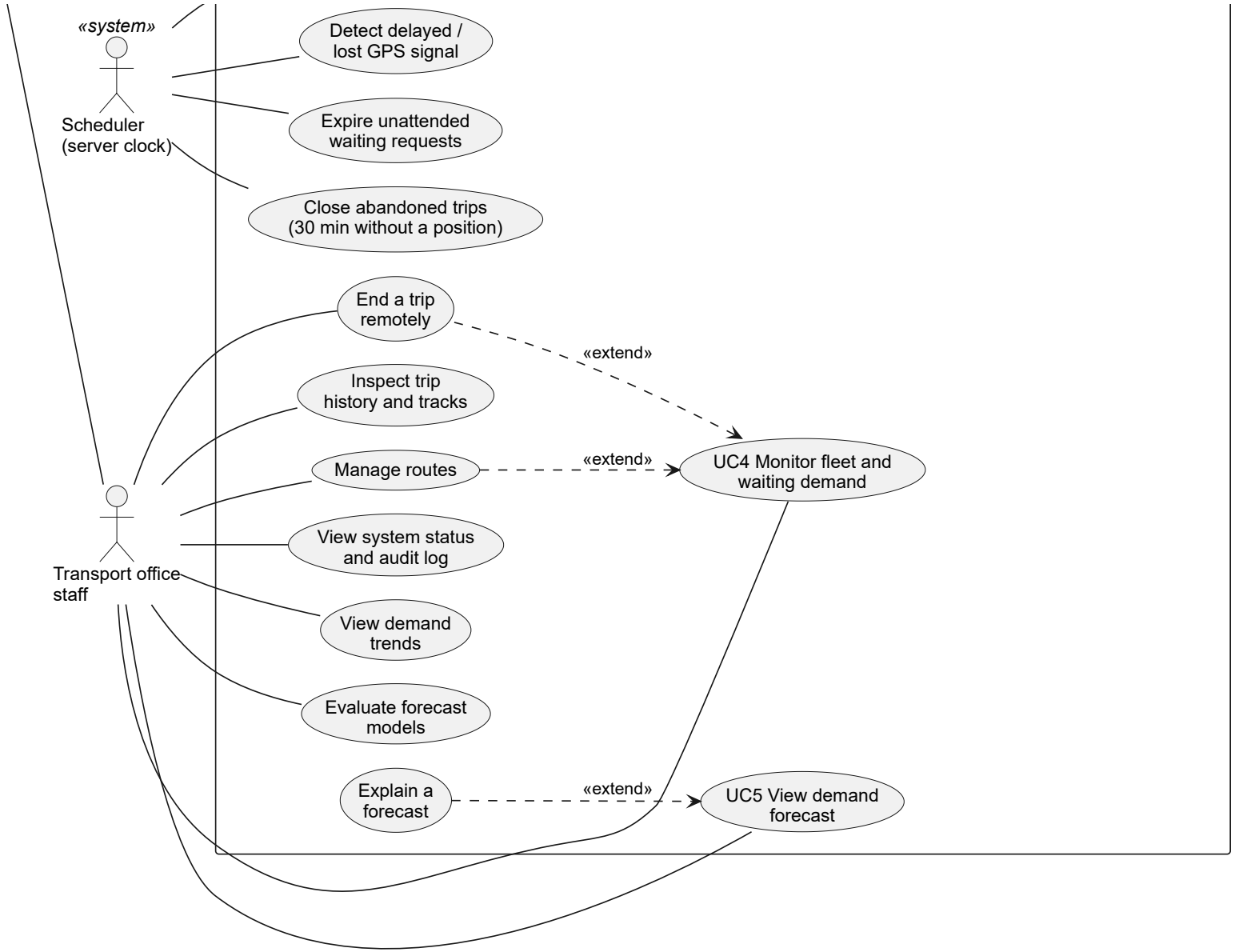


Figure A.1 — Use case diagram — detail 2 of 2 (row 2 of 2), at 83 %

Three primary human actors (Student, Driver, Transport office staff) share a generalised *Campus user* actor for sign-in. Two system actors appear as secondary actors: the **GPS receiver** (the phone's GPS or the built-in simulator) and the **Scheduler** (the server clock that detects stale GPS and expires waiting requests). `«include»` marks behaviour that always happens as part of a use case; `«extend»` marks optional behaviour.

A.2 Use case descriptions

The five primary use cases below are all implemented end to end in the prototype. UC1 and UC2 together form the main demonstration scenario.

UC1 — Track shuttles and view arrival times

Field	Description
Primary actor	Student
Stakeholders and interests	Student: wants to know whether to walk to the stop now, without waiting in the heat. Transport office: fewer phone calls asking "where is the bus?". Driver: wants passengers ready at the stop.
Preconditions	Student is signed in. At least one route is in service.
Trigger	Student opens the student page.
Main success scenario	1. System shows the schematic map with routes, stops and every active shuttle, and opens a live feed. 2. Student selects a stop (on the map or from a list; the choice is remembered on that device). 3. System lists every shuttle on a route serving that stop, soonest first, with the estimated minutes, clock time, distance, passenger load and a live-signal badge. 4. As drivers send positions, the system pushes updates; positions and minutes change without reloading. 5. When a shuttle on a loop route reaches the selected stop, its next-lap estimate is recalculated and the list is sorted again. A non-loop route does not invent a next lap.
Extensions	3a. No shuttle is running on a route serving the stop: system says so plainly. 4a. A shuttle's newest position is more than 20 s old: its estimate is shown as approximate ("~6 min") with "location delayed"; when the estimate time has passed it shows "Due". 4b. No position for more than 60 s: the system shows "GPS signal lost · last seen N min ago" and **no estimate**. 4c. The student's connection drops: a banner says "reconnecting — times may be out of date"; on reconnection a full snapshot replaces the old state. 4d. Session expired: banner offers to sign in again.
Postconditions	None (read-only use case).
Quality attributes	Freshness (QA-1), scalability with many viewers (QA-2), honesty under GPS loss (QA-3), privacy: no driver identity is ever sent to students (QA-6).
Implemented in	`public/js/student.js`, `src/web/routes/shared-routes.ts`, `src/realtime/sse-hub.ts`, `src/tracking/tracking-service.ts`

UC2 — Run a trip (start, share location, end)

Field	Description
Primary actor	Driver. Secondary actor: GPS receiver (device GPS or simulator).
Stakeholders and interests	Driver: minimal interaction while working. Students: accurate positions. Office: knows which vehicle is on which route and who is accountable.
Preconditions	Driver is signed in, has no other active trip, and the chosen shuttle is free.
Trigger	Driver taps "Start trip" after choosing route, shuttle and location source.

Field	Description
Main success scenario	1. System validates the route is in service and that the shuttle and driver are free, then creates an ACTIVE trip (signal "awaiting first GPS position") and announces it to all viewers. 2. Driver app starts the GPS source. 3. The simulator obtains a position every ~3 s (device GPS is event-driven and throttled to at most one fix per 2 s) and uploads all queued positions in one batch with the device's current time. 4. System corrects device clock skew, validates and orders the positions, stores them, snaps the shuttle onto the route, updates its smoothed speed, recalculates ETAs, detects stops reached, and pushes one update to all viewers. 5. App shows "what passengers see" (Live / Delayed / GPS lost) and the next stop. 6. Driver optionally sets passenger load (seats available / filling up / full). 7. Driver taps "End trip" and confirms; the app first delivers the queued positions (waiting at most 5 s), holds new ones and sends the end request; the system completes the trip and removes the shuttle from every map; only after the server confirms does the app stop GPS and show "Trip ended".
Extensions	1a. Shuttle or driver already on an active trip, or route out of service: system refuses with the reason (HTTP 409; an unknown route or shuttle gives 404). 3a. GPS produces no positions (tunnel, outage): nothing is uploaded; students see the delayed → lost sequence (UC1 4a–4b). 3b. Network unavailable: positions stay queued in the open driver page (newest 500 kept; older ones dropped and counted; lost if the page is closed or reloaded) and are retried with exponential backoff (1 s → 15 s); when the network returns they are delivered in batches of up to 50 and the shuttle jumps to its true position; positions more than 10 minutes old on arrival are rejected. 4a. A position is duplicated, out of order, physically impossible (> 110 km/h jump), too inaccurate (> 150 m), too old (> 10 min) or outside the service area: it is rejected with a reason code and the rest of the batch is still accepted. 4b. The page is reloaded: the trip resumes from the server's last known position with the same location source as its latest accepted position (device GPS or simulator). If no position has been accepted yet, the app shares nothing until the driver chooses a source. 7a. The office ended the trip remotely (UC4): the next upload returns 409/404 and the app returns to the start screen with an explanation. 7b. No accepted position for 30 minutes (counted from the last position, or from the trip start if there was none): the system closes the trip automatically (TERMINATED, reason NO_SIGNAL_TIMEOUT). 7c. The end request fails (no network, expired session, server unreachable): the trip stays ACTIVE, location sharing continues, the app says the trip was **not** ended and the driver can try again; if the server answers that the trip had already ended (409/404), the app treats it as ended.
Postconditions	The trip is COMPLETED or TERMINATED, with its GPS trace stored (deleted after 30 days).
Implemented in	`public/js/driver.js`, `public/js/gps.js`, `src/web/routes/driver-routes.ts`, `src/tracking/*`

UC3 — Register a waiting request ("I'm waiting here")

Field	Description
Primary actor	Student
Stakeholders and interests	Student: wants the shuttle to know someone is waiting. Office: real-time demand signal and history for forecasts. Privacy officer: no tracking of individual students.
Preconditions	Student is signed in and has selected a stop.
Main success scenario	1. Student taps "I'm waiting here", optionally choosing a specific route. 2. System records the request against a **pseudonymous reference** (keyed hash of the user id), sets a 20-minute expiry, and pushes the stop's new waiting count to all viewers. 3. The student sees "You're on the waiting list at Library · expires in 20 min". 4. When a shuttle on a matching route reaches the stop, the system marks the request SERVED and the student sees "A shuttle reached Library at 08:14".

Field	Description
Extensions	1a. Same stop and route requested again: the existing request is returned (idempotent). 1b. Different stop or route: the previous request is cancelled and replaced. 1c. Route does not serve the stop: rejected with a reason. 1d. More than 6 requests in an hour: rejected (429) to prevent abuse that would distort demand data. 3a. Student cancels: request CANCELLED and the count updates. The open page reconciles its private request after a matching waiting-count event and after a reconnect snapshot, including closures missed while offline. 4a. No shuttle within 20 minutes: request EXPIRED.
Postconditions	Request closed in one of SERVED / CANCELLED / EXPIRED; 24 h later its pseudonymous link is erased and it remains only as anonymous demand data.
Implemented in	`src/demand/waiting-service.ts`, `src/web/routes/student-routes.ts`, `public/js/student.js`

UC4 — Monitor the fleet and waiting demand (with remote trip termination)

Field	Description
Primary actor	Transport office staff
Preconditions	Staff member signed in with the office role.
Main success scenario	1. System shows KPI tiles (shuttles on trip, GPS problems, students waiting, live viewers), the fleet map with waiting badges, waiting counts per stop, and a table of active trips including the accountable driver, GPS state, last position age, load and next-stop ETA — all updated live. 2. Staff opens a trip's track to inspect its GPS trace: positions, distance, duration, longest gap and number of gaps longer than the stale threshold.
Extensions	1a. *End a trip remotely* («extend»): staff chooses "End", must give a reason; the system terminates the trip, records actor, time and reason in the audit log, and the driver app is told on its next contact. 1b. *Manage routes*: take a route out of service or back, edit name/colour/planned speed/dwell time, or create a new route from an ordered list of stops (created inactive for review). Every change is audited.
Implemented in	`public/js/office.js`, `src/web/routes/office-routes.ts`, `src/network/network-service.ts`

UC5 — View the demand forecast (with explanation)

Field	Description
Primary actor	Transport office staff
Main success scenario	1. Staff opens "Demand & forecast", picks a date and a model. 2. System shows the expected requests per hour (all stops), a stop × hour heat map, the peak hour and busiest stop, and states which data sources were used (synthetic and/or live). 3. Staff selects a cell; the system explains it: method, the exact past observations used (dates, counts, weights) and the plausible range. 4. Staff compares models in the backtest table (MAE, RMSE, bias, range coverage).
Extensions	2a. Not enough history for a stop-hour: the cell shows "–" and **no number is invented**. 2b. History contains synthetic data: a warning banner states that the figures illustrate the method only.
Implemented in	`src/demand/demand-service.ts`, `src/demand/prediction/*`, `public/js/office.js`

A.3 Activity diagram — Run a trip

Swimlanes separate the driver app from the server. The two interruption paths are visible: *GPS outage* (no position is appended to the queue) and *network loss* (the queue is kept and retried with backoff). Two further paths are shown: the server ending the trip while it runs (office or 30-minute auto-close), and ending the trip, where the outer loop repeats until the server confirms the end, so a failed request leaves the trip active.

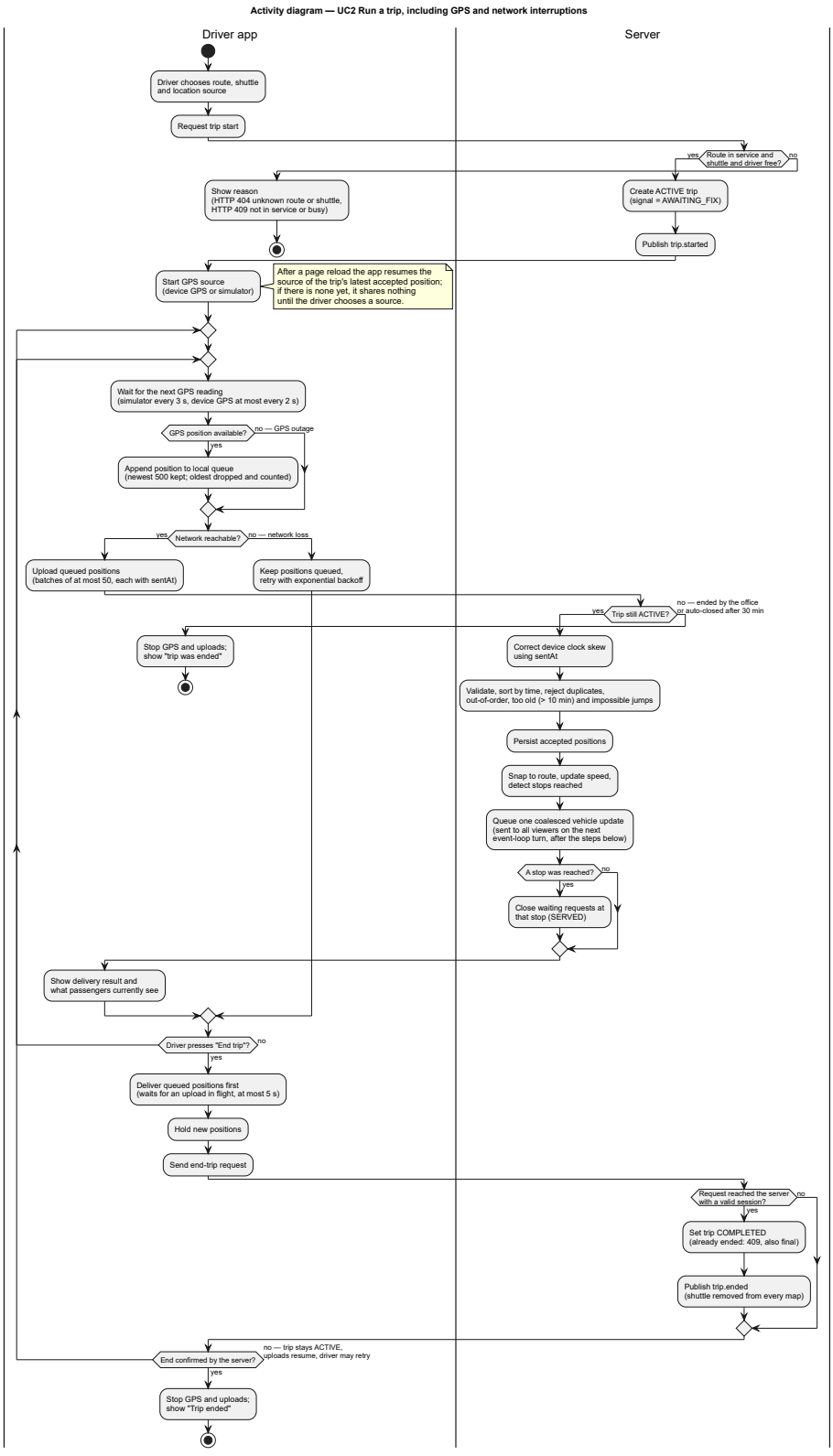


Figure A.3 — Activity diagram: Run a trip with GPS and network interruptions — overview at 37 % (read the 3 detail pages that follow, at 75 %; on screen, zoom in)

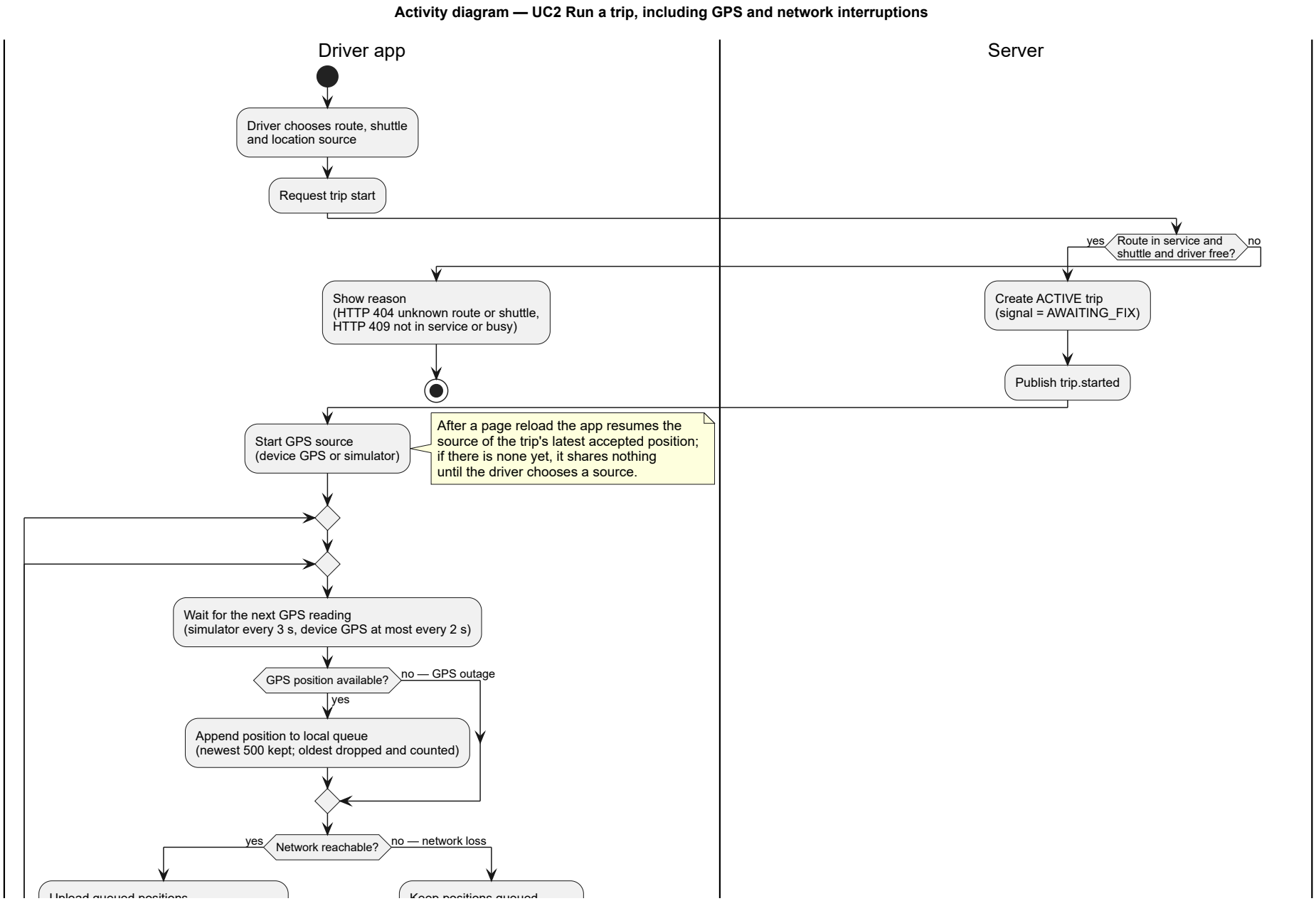


Figure A.3 — Activity diagram: Run a trip with GPS and network interruptions — detail 1 of 3 (row 1 of 3), at 75 %

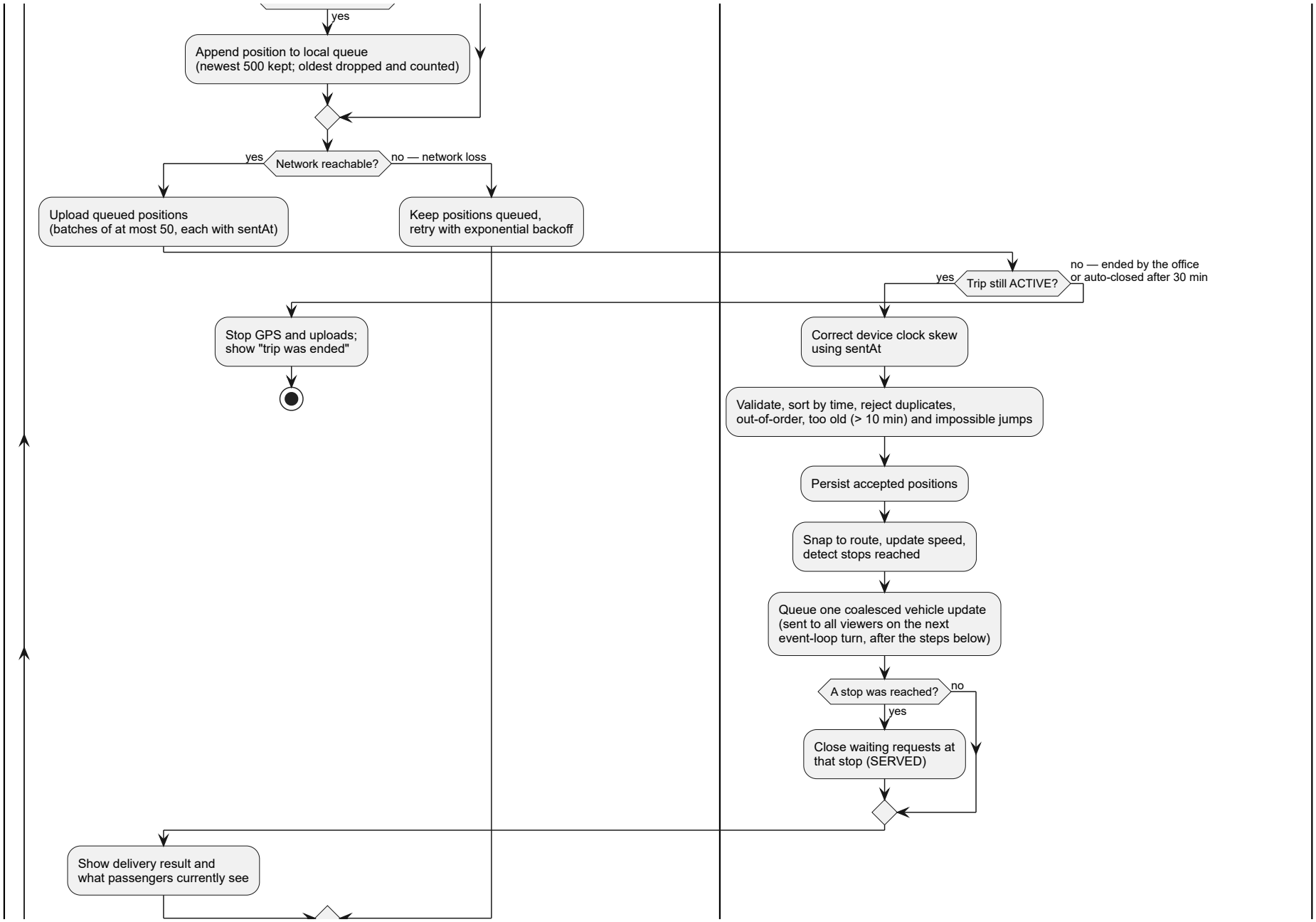


Figure A.3 — Activity diagram: Run a trip with GPS and network interruptions — detail 2 of 3 (row 2 of 3), at 75 %

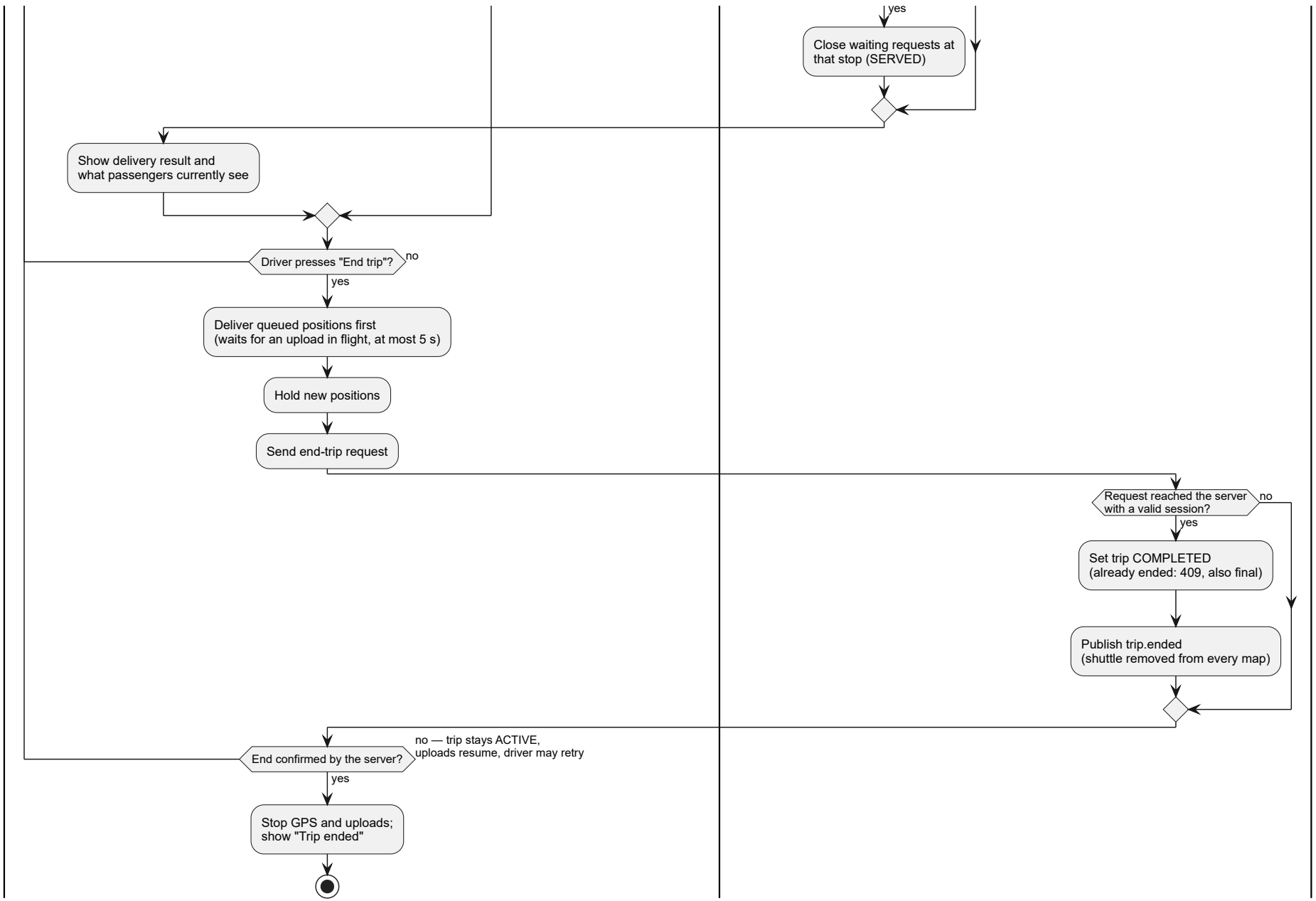


Figure A.3 — Activity diagram: Run a trip with GPS and network interruptions — detail 3 of 3 (row 3 of 3), at 75 %

A.4 System sequence diagram — Run a trip and track it

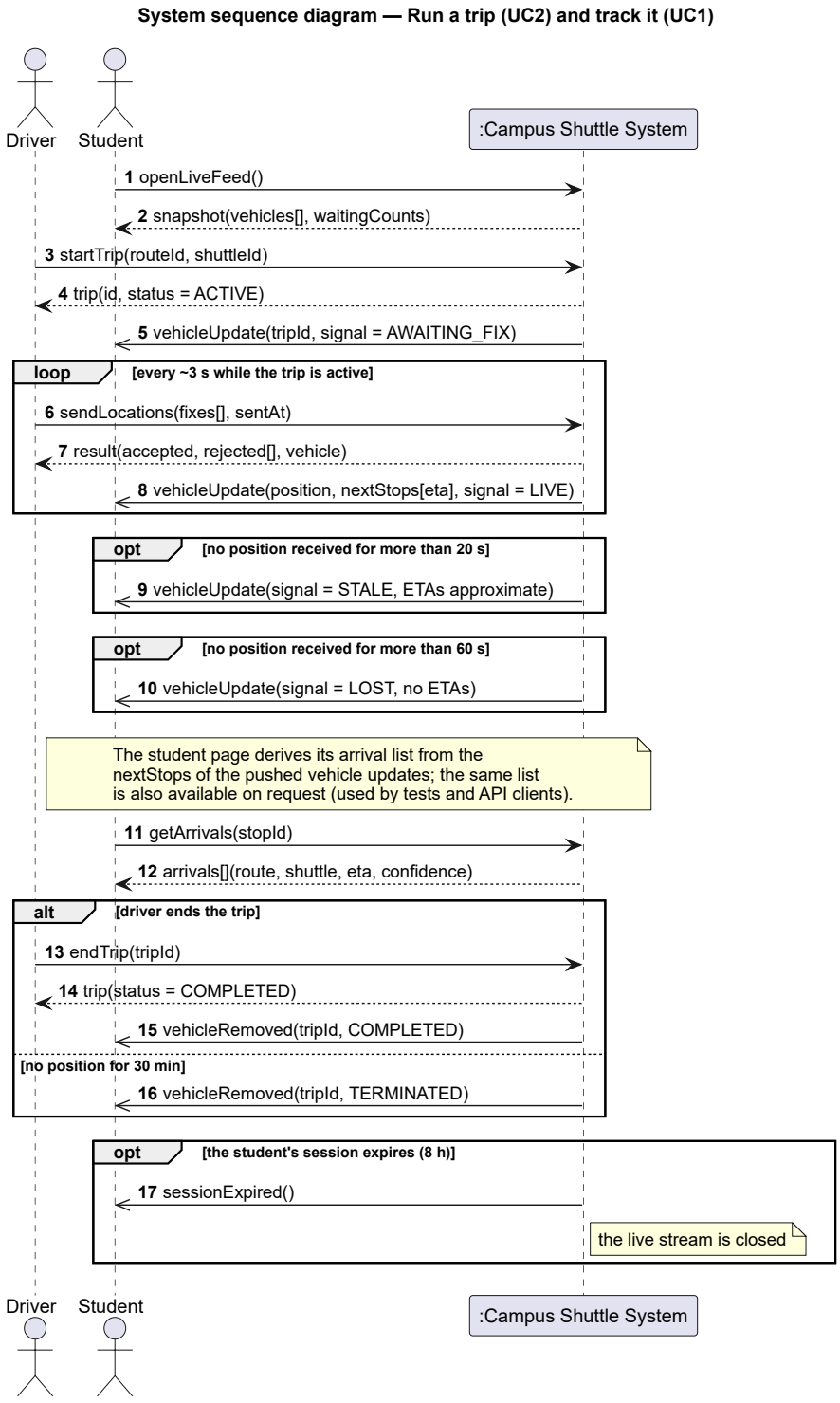


Figure A.4 — System sequence diagram for UC2 + UC1

The system is a black box; messages are the system operations of the REST API and the live feed. The `opt` fragments are system-initiated (time) events.

A.5 Design sequence diagram — processing one location upload

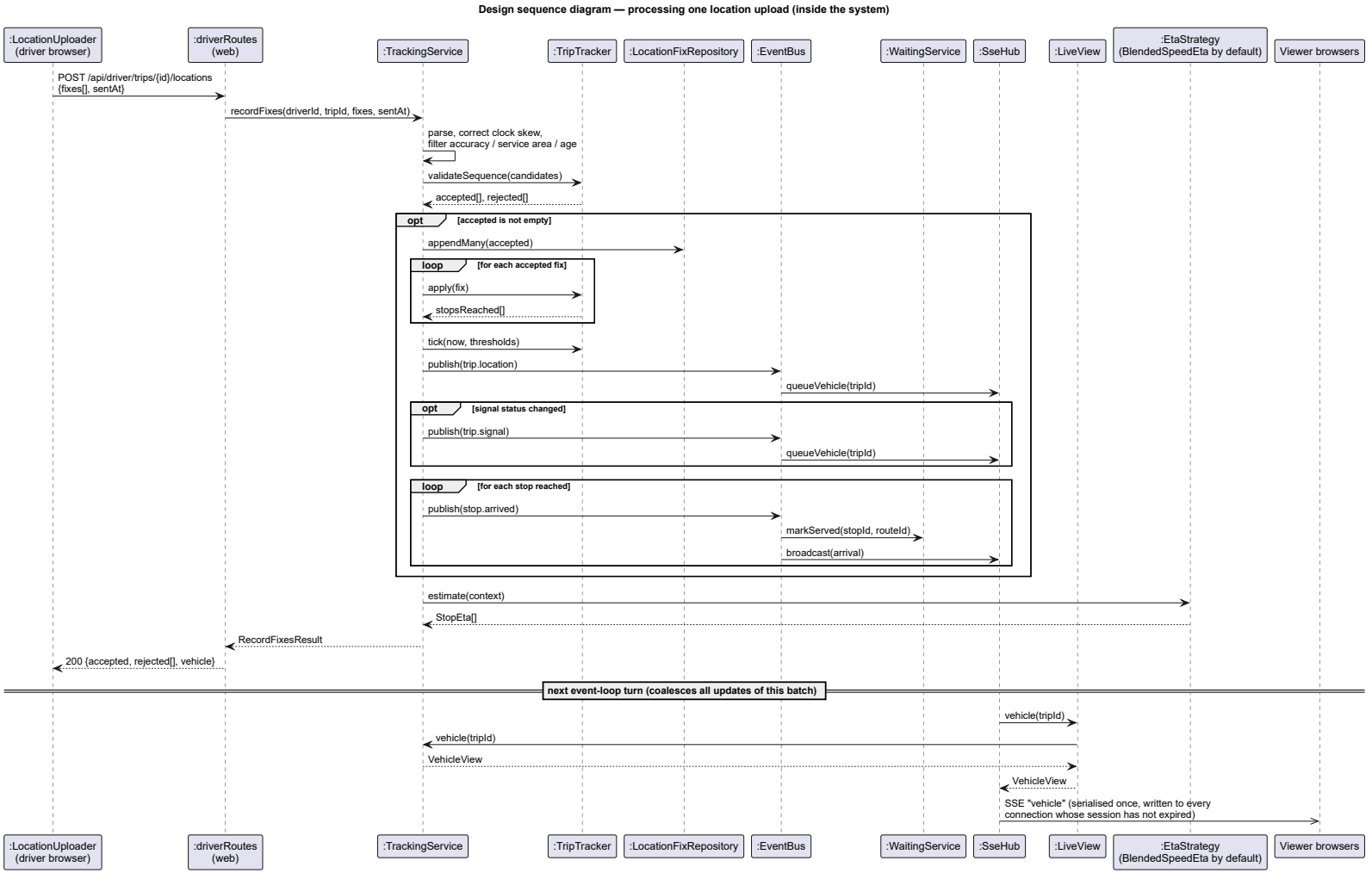


Figure A.5 — Inside the system: one location upload — overview at 45 % (read the 2 detail pages that follow, at 67 %; on screen, zoom in)

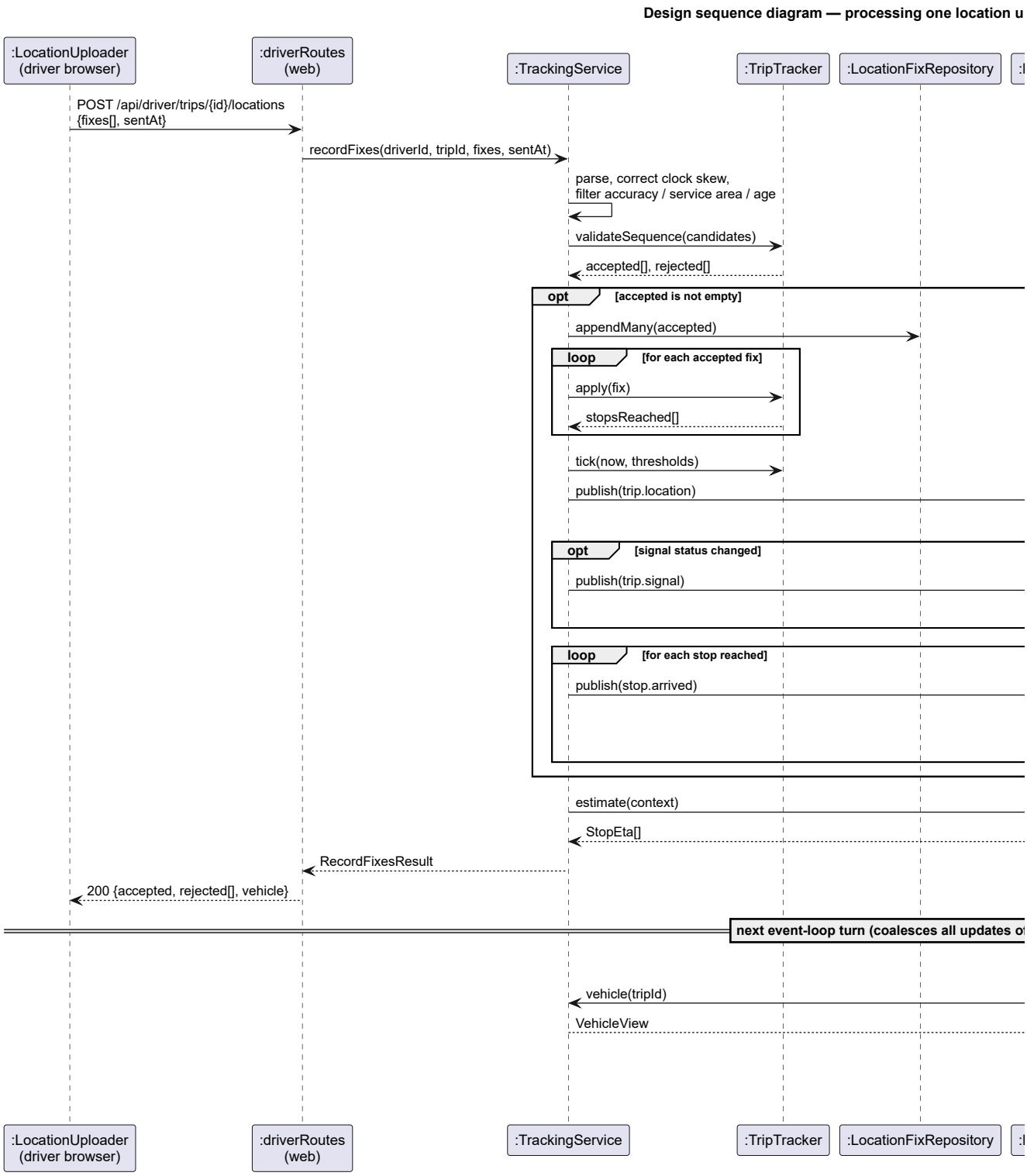


Figure A.5 — Inside the system: one location upload — detail 1 of 2 (column 1 of 2), at 67 %

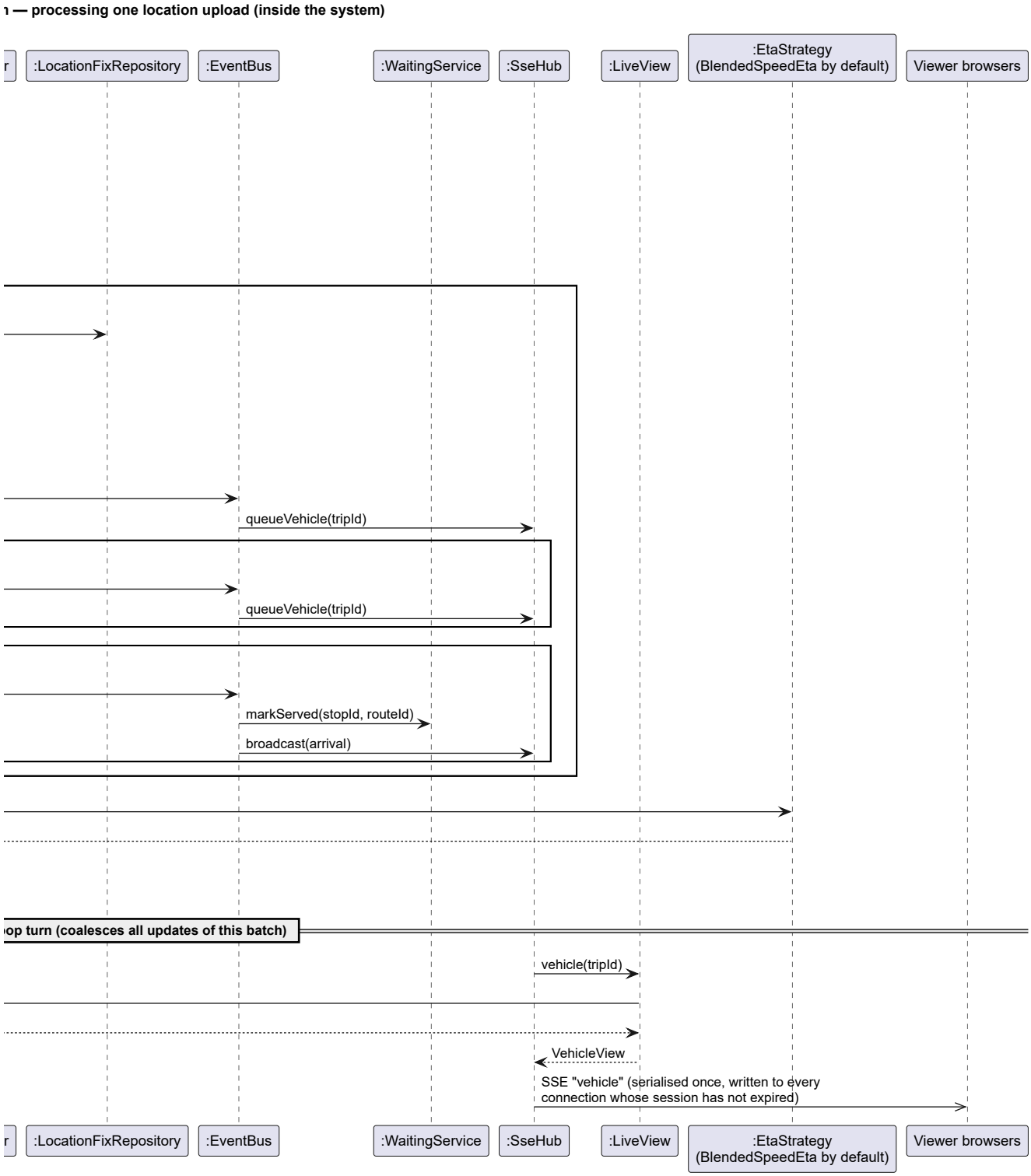


Figure A.5 — Inside the system: one location upload — detail 2 of 2 (column 2 of 2), at 67 %

A.6 State machine — Trip and GPS signal

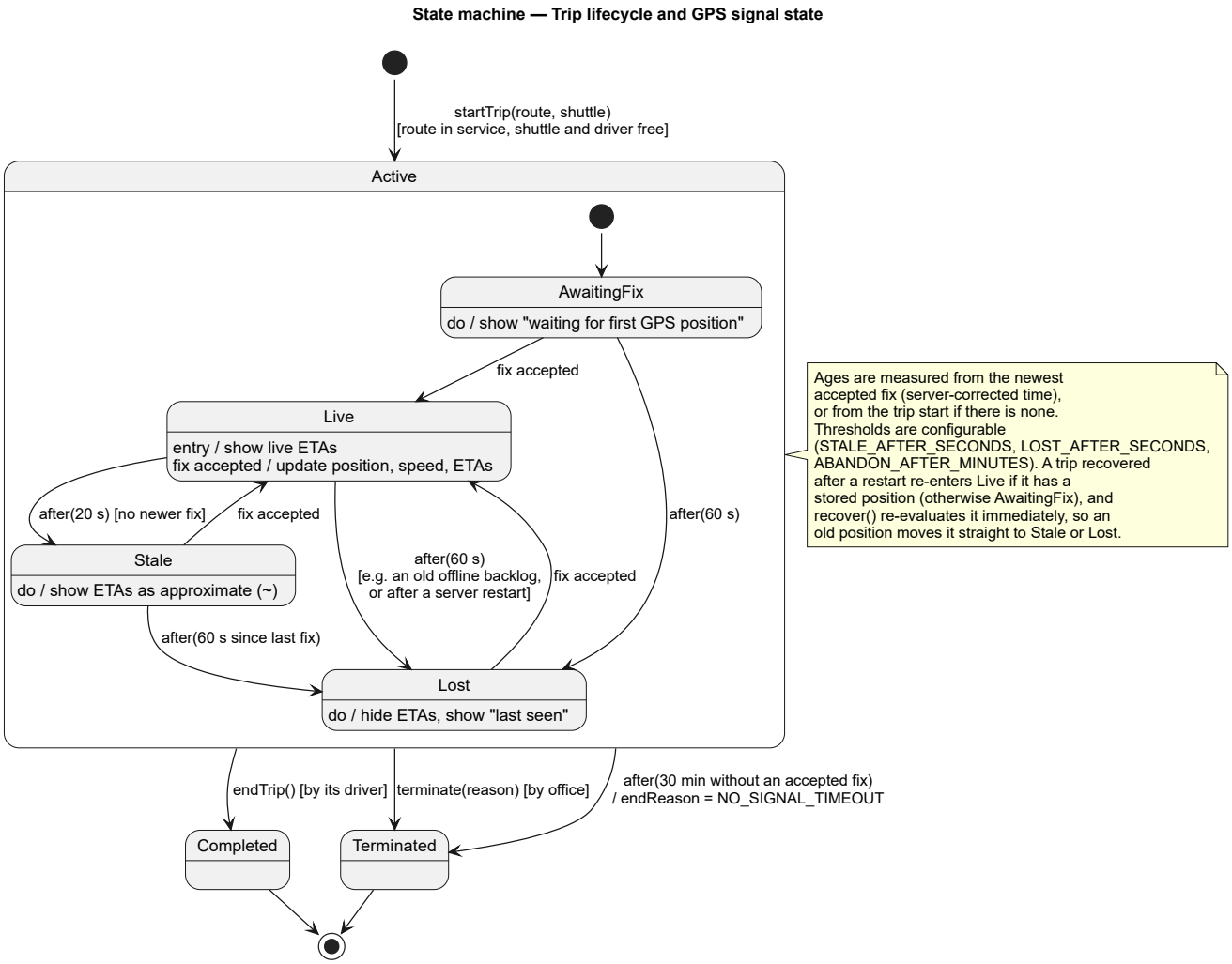


Figure A.6 — State machine for a trip and its GPS signal

`Active` is a composite state; its sub-states are implemented one-to-one by the State pattern classes in `src/tracking/signal-state.ts` (Figure A.15). Time events use UML `after(...)` notation.

A.7 State machine — Waiting request

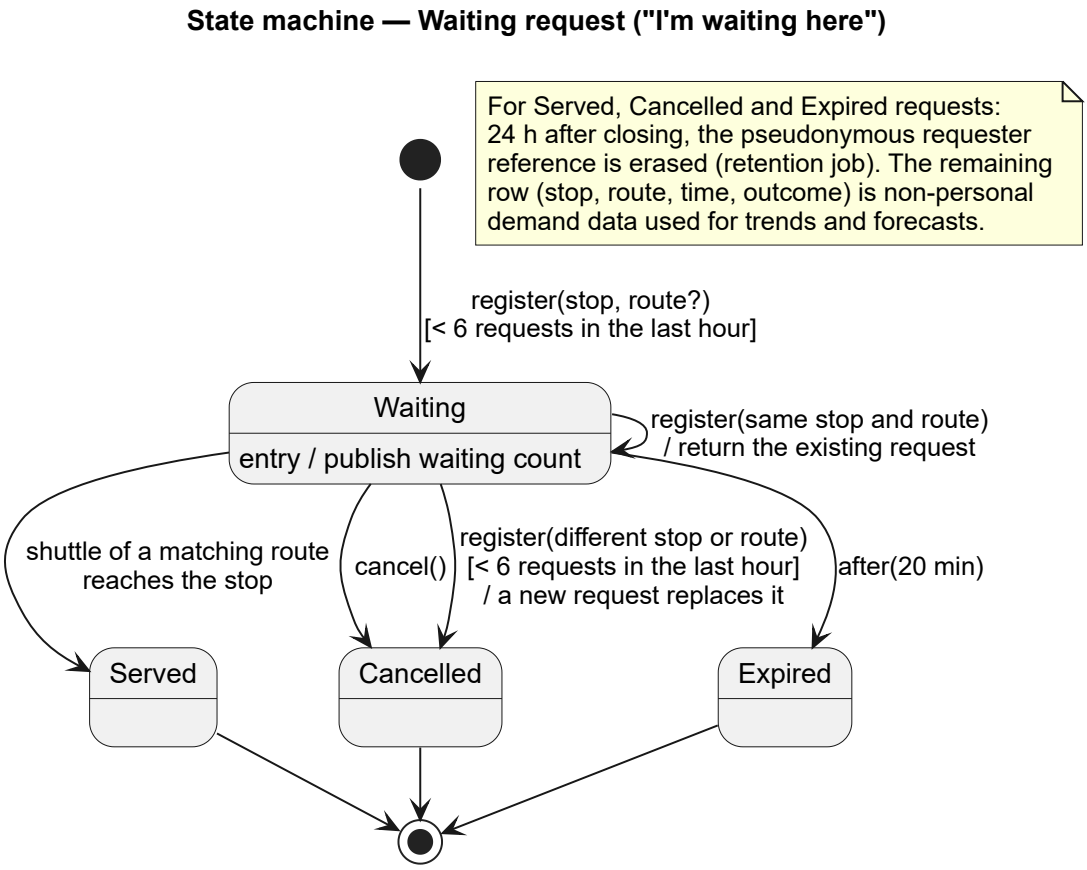


Figure A.7 — State machine for a waiting request

A.8 Domain model

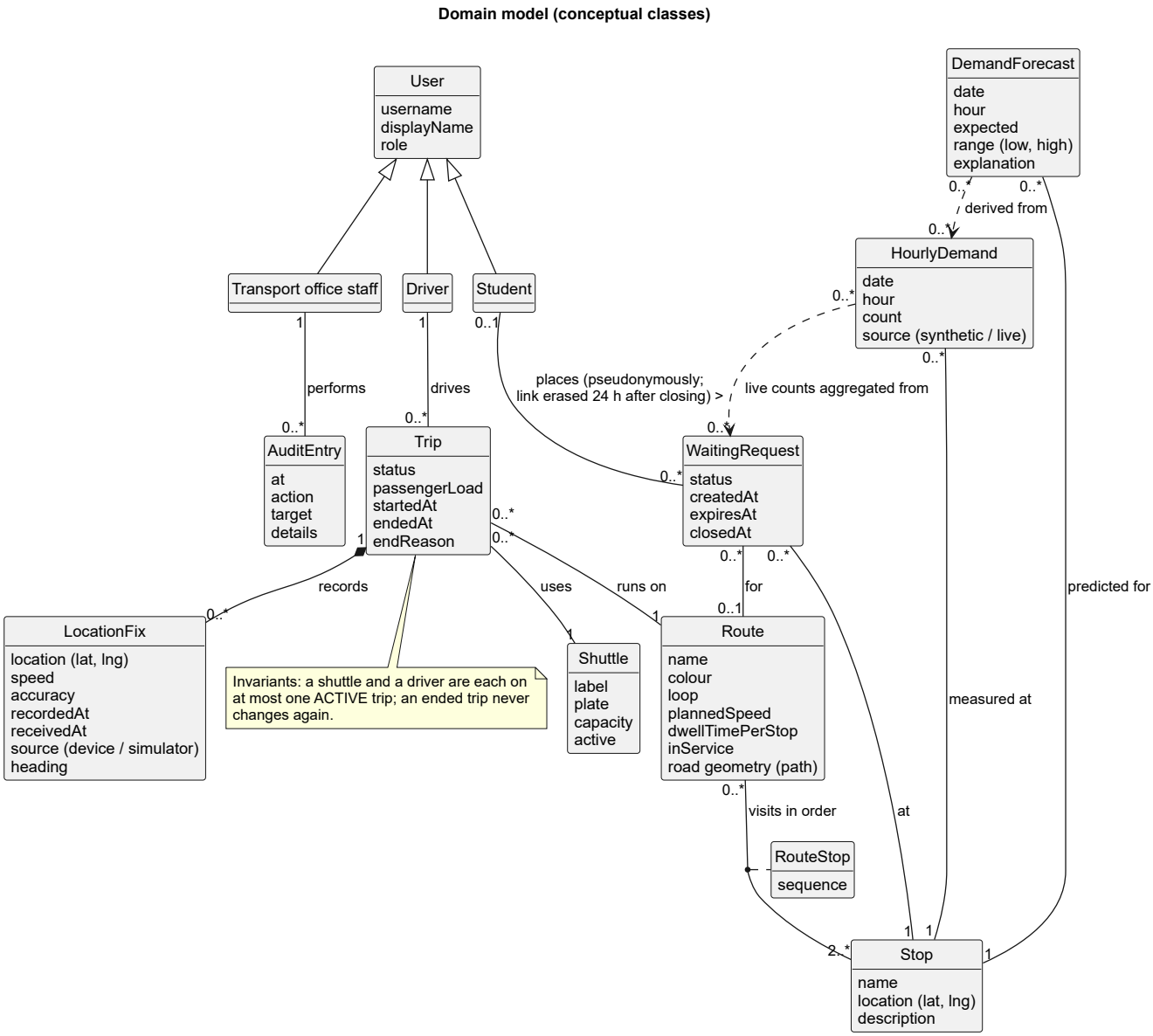


Figure A.8 — Domain model

Conceptual classes only (no operations). `RouteStop` is an association class carrying the stop order of a route.

A.9 Design class diagram 1 of 2 — tracking, network, real-time

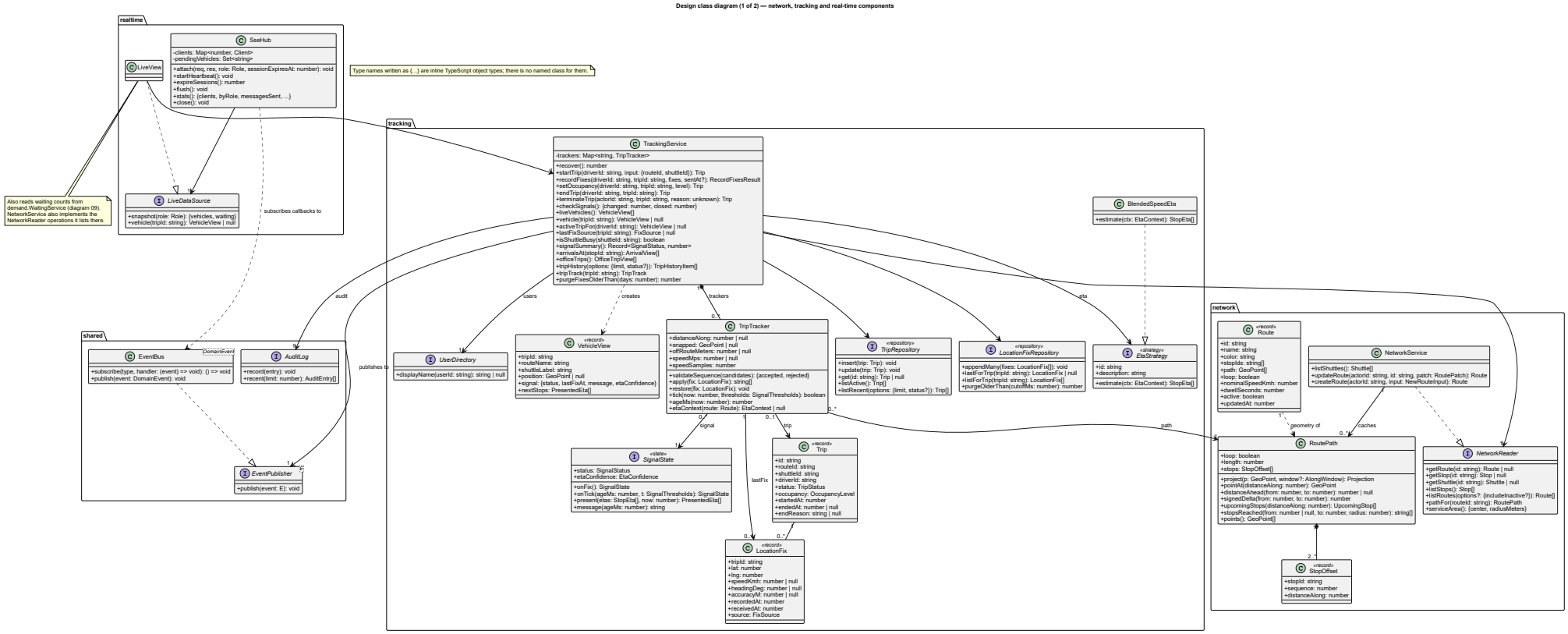


Figure A.9 — Design class diagram: tracking, network and real-time — overview at 28 % (read the 4 detail pages that follow, at 63 %; on screen, zoom in)

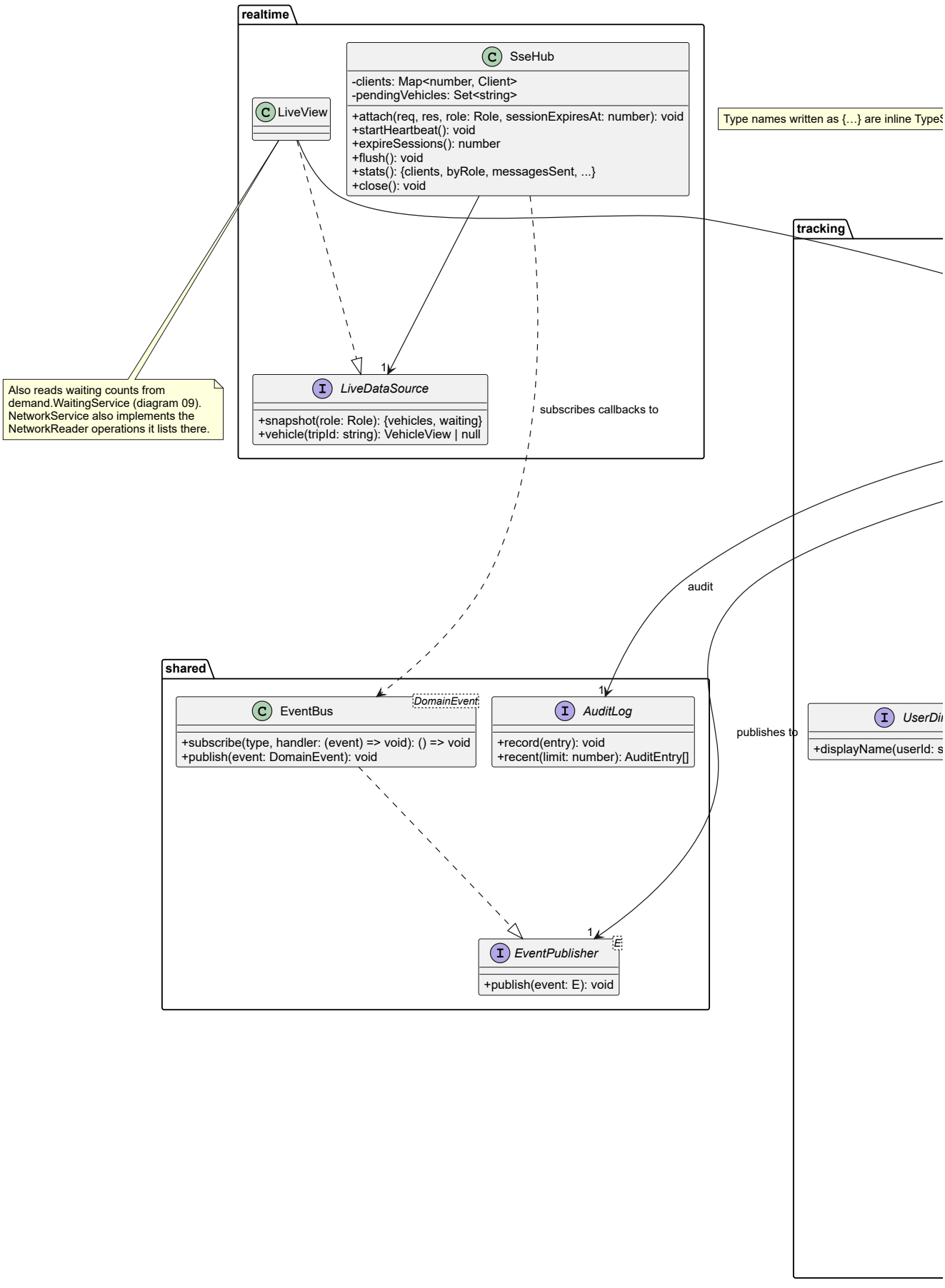


Figure A.9 — Design class diagram: tracking, network and real-time — detail 1 of 4 (column 1 of 4), at 63 %

mes written as {…} are inline TypeScript object types; there is no named class for them.

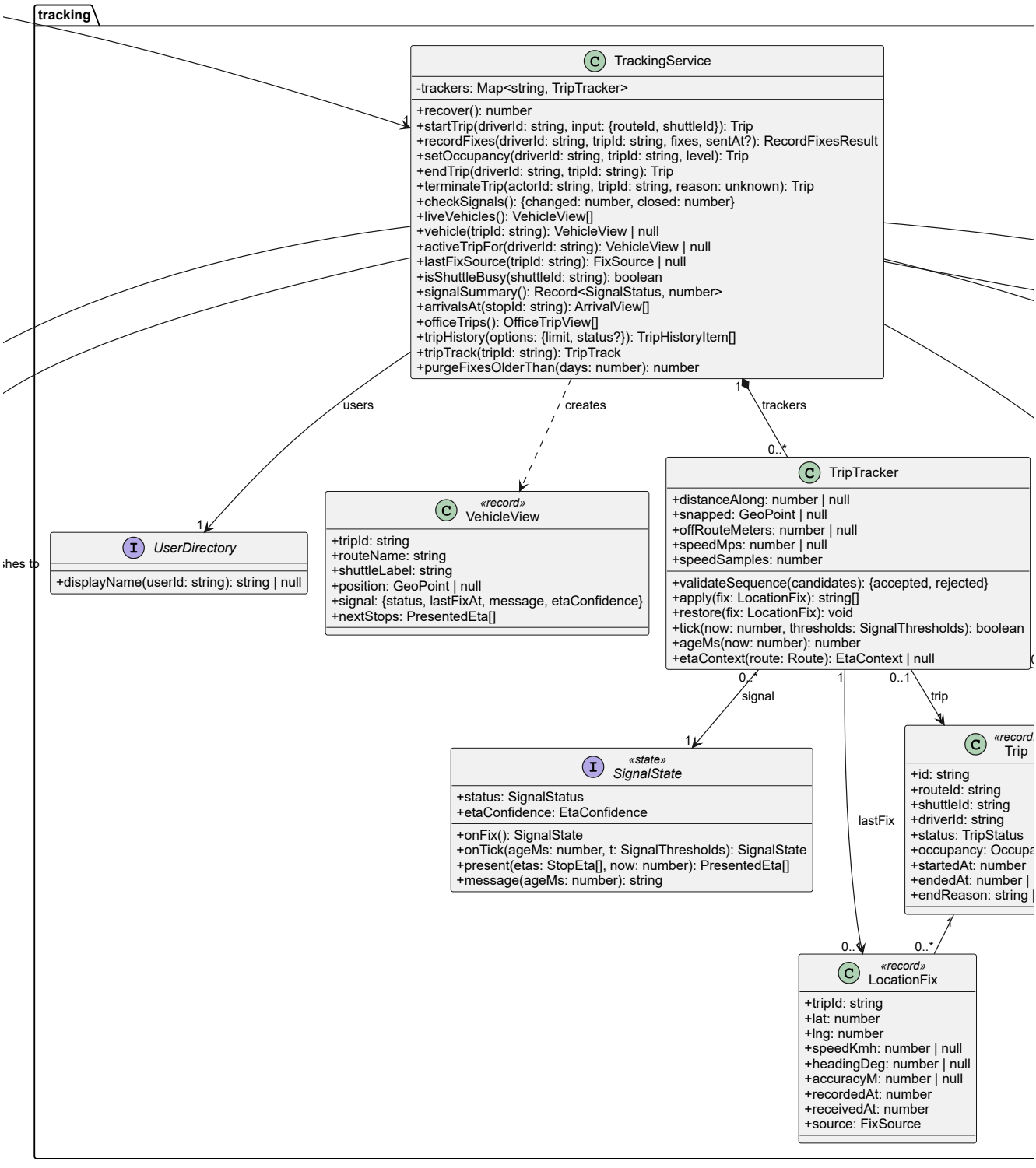


Figure A.9 — Design class diagram: tracking, network and real-time — detail 2 of 4 (column 2 of 4), at 63 %

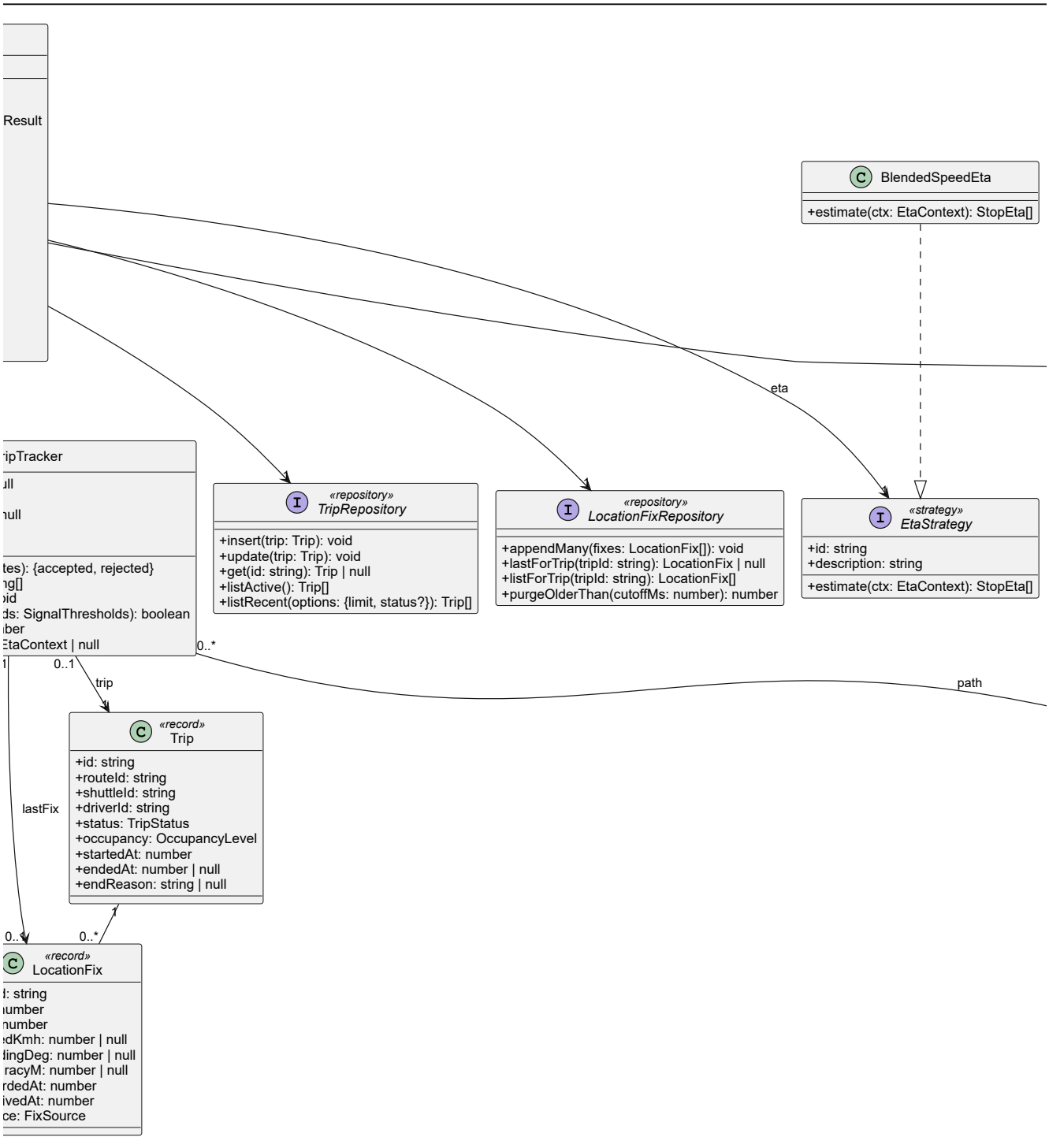


Figure A.9 — Design class diagram: tracking, network and real-time — detail 3 of 4 (column 3 of 4), at 63 %

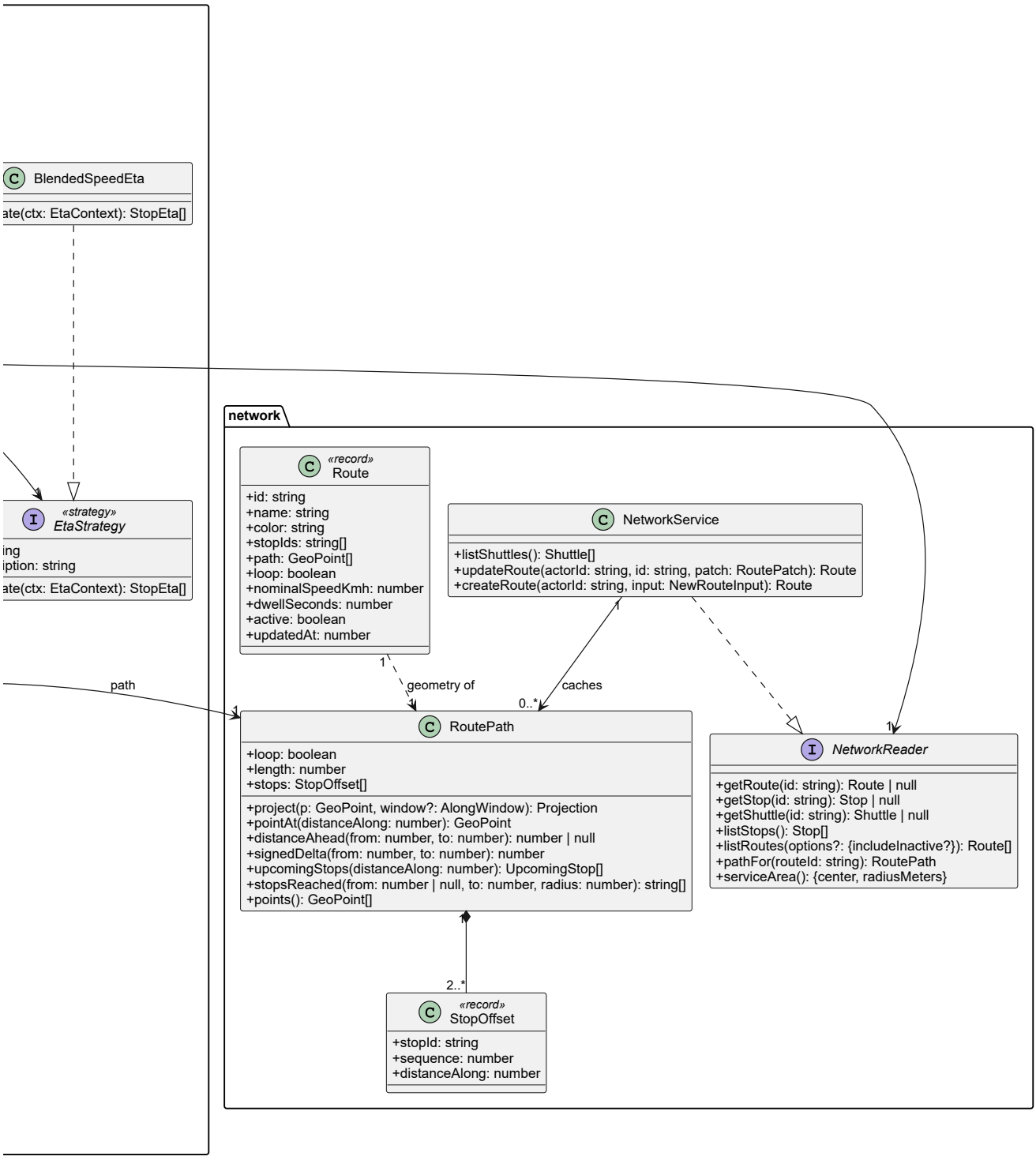


Figure A.9 — Design class diagram: tracking, network and real-time — detail 4 of 4 (column 4 of 4), at 63 %

A.10 Design class diagram 2 of 2 — demand, forecasting, identity

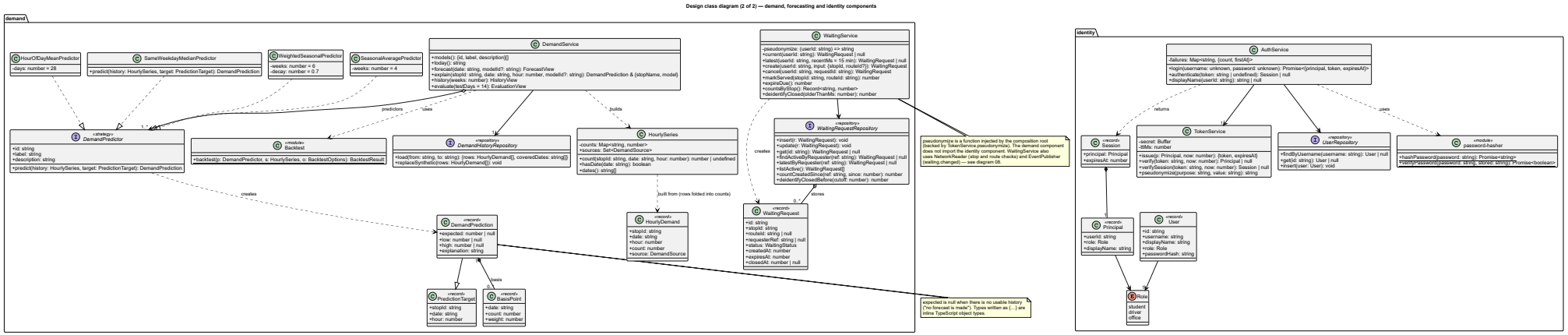


Figure A.10 — Design class diagram: demand, forecasting and identity — overview at 24 % (read the 3 detail pages that follow, at 65 %; on screen, zoom in)

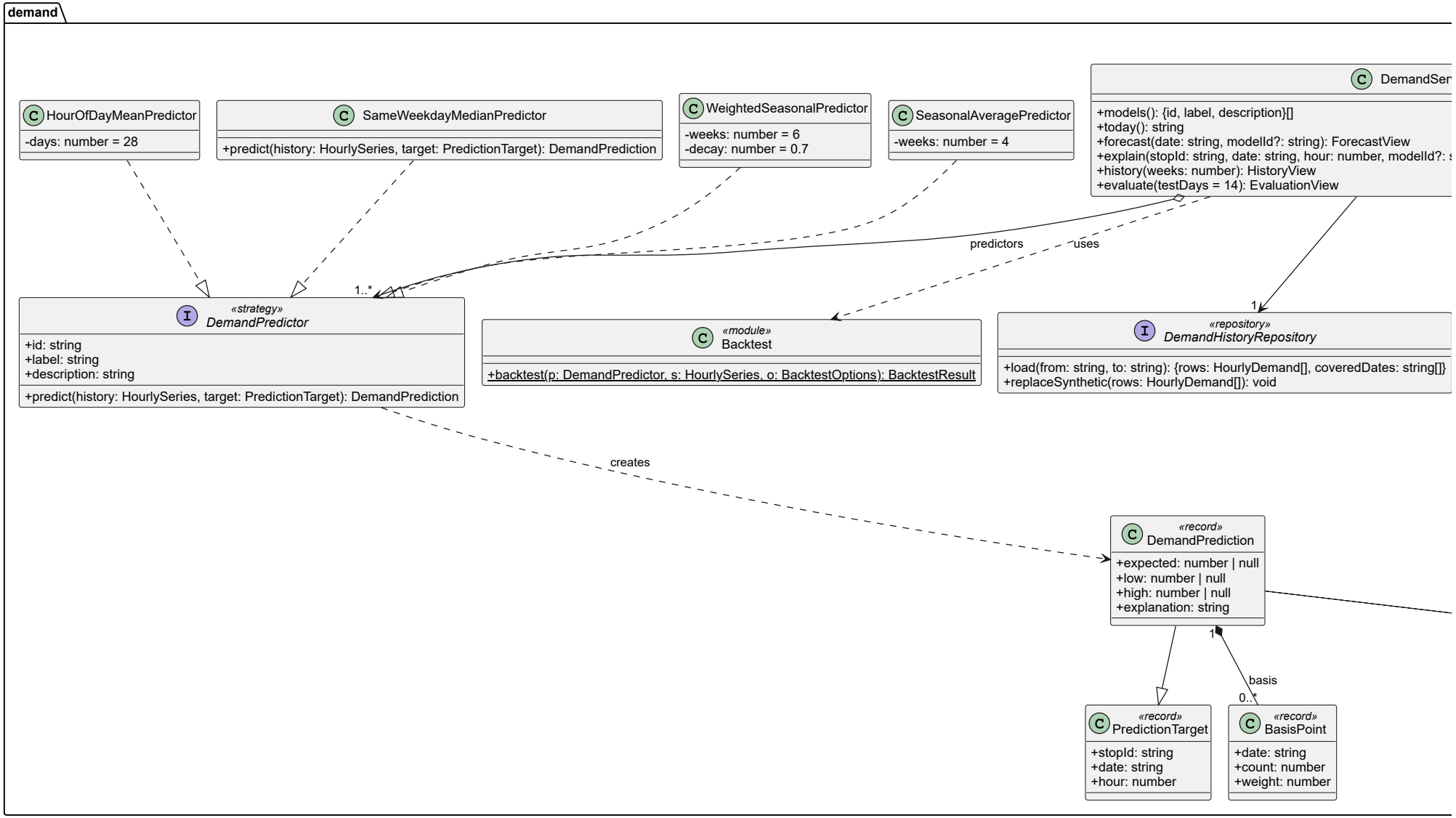


Figure A.10 — Design class diagram: demand, forecasting and identity — detail 1 of 3 (column 1 of 3), at 65 %

Design class diagram (2 of 2) — demand, forecasting and identity components

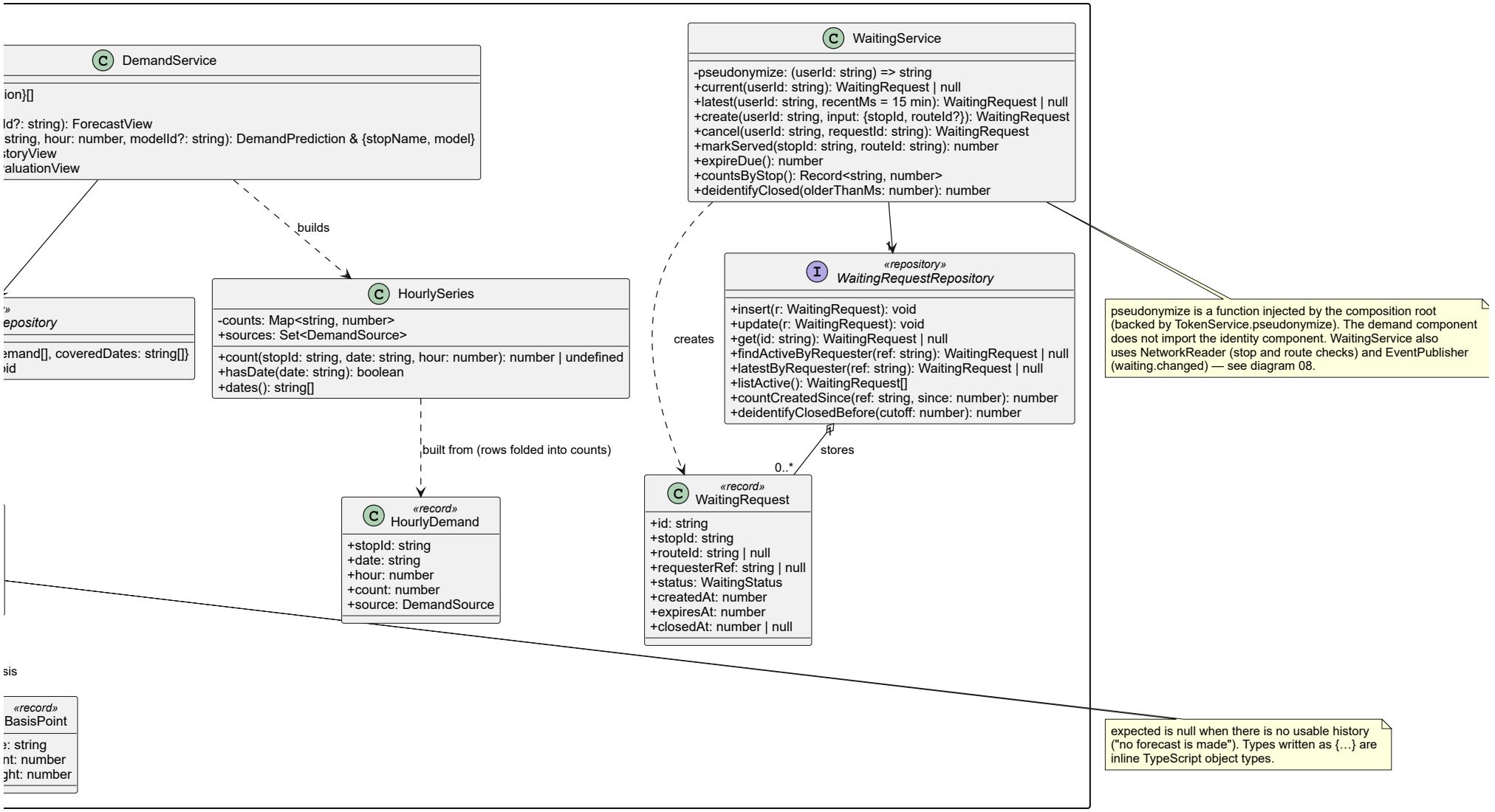


Figure A.10 — Design class diagram: demand, forecasting and identity — detail 2 of 3 (column 2 of 3), at 65 %

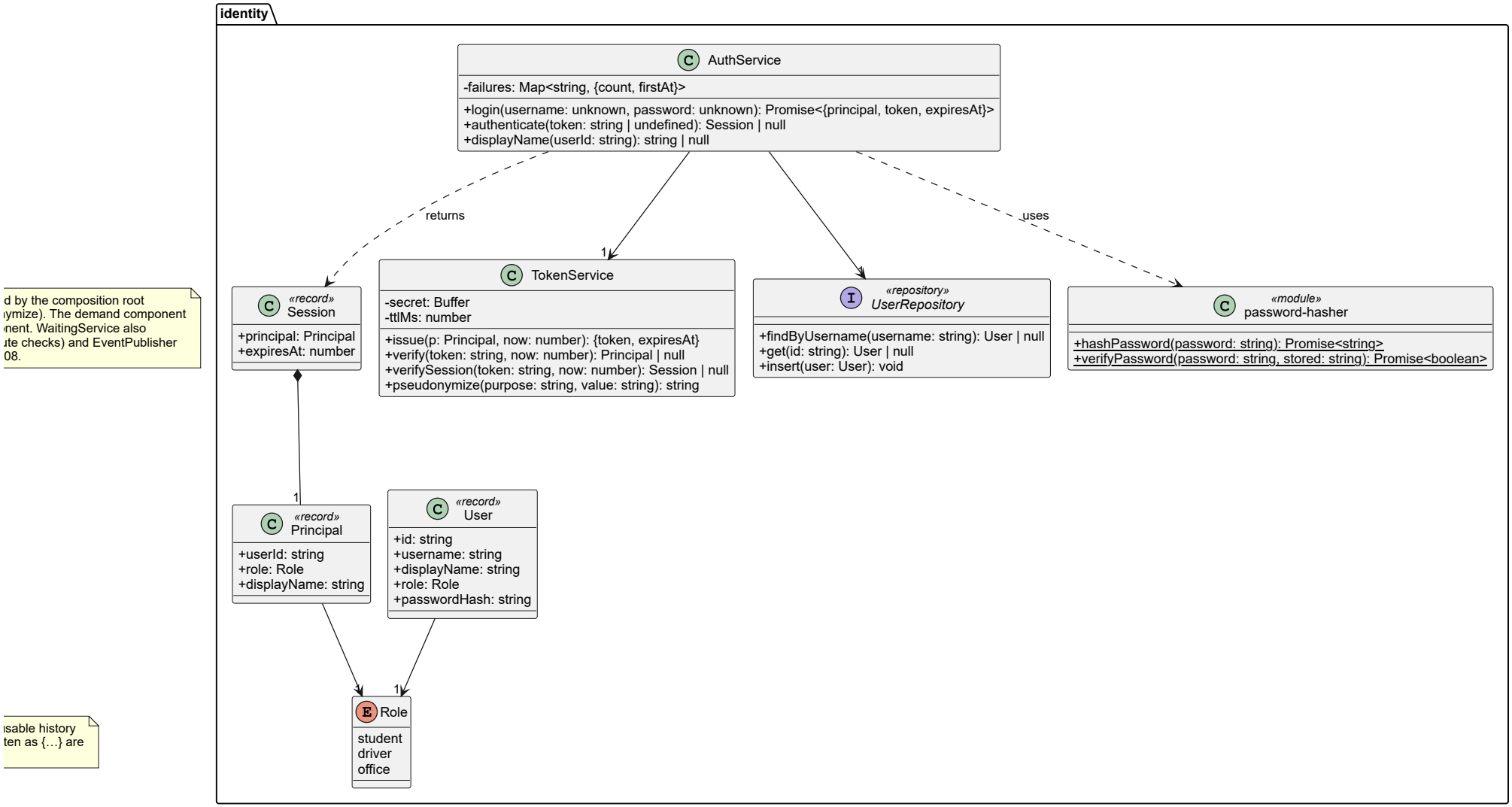


Figure A.10 — Design class diagram: demand, forecasting and identity — detail 3 of 3 (column 3 of 3), at 65 %

A.11 Component diagram

Each component is a folder under `src/` with a public service class and port interfaces. Provided interfaces are drawn as lollipops, required ones as dependencies on them.

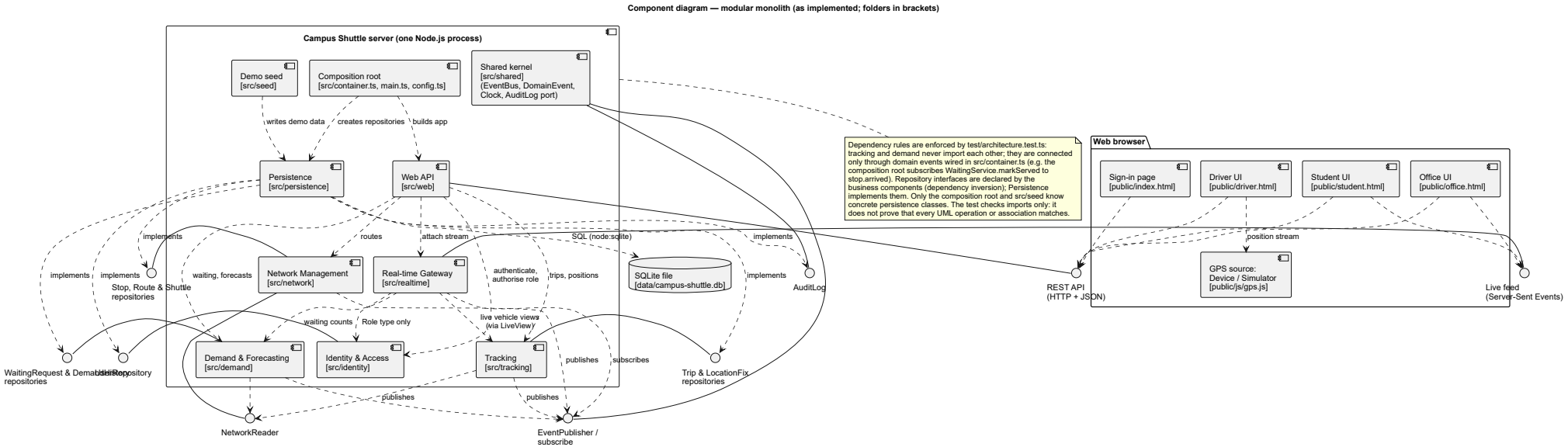


Figure A.11 — Component diagram — overview at 38 % (read the 2 detail pages that follow, at 71 %; on screen, zoom in)

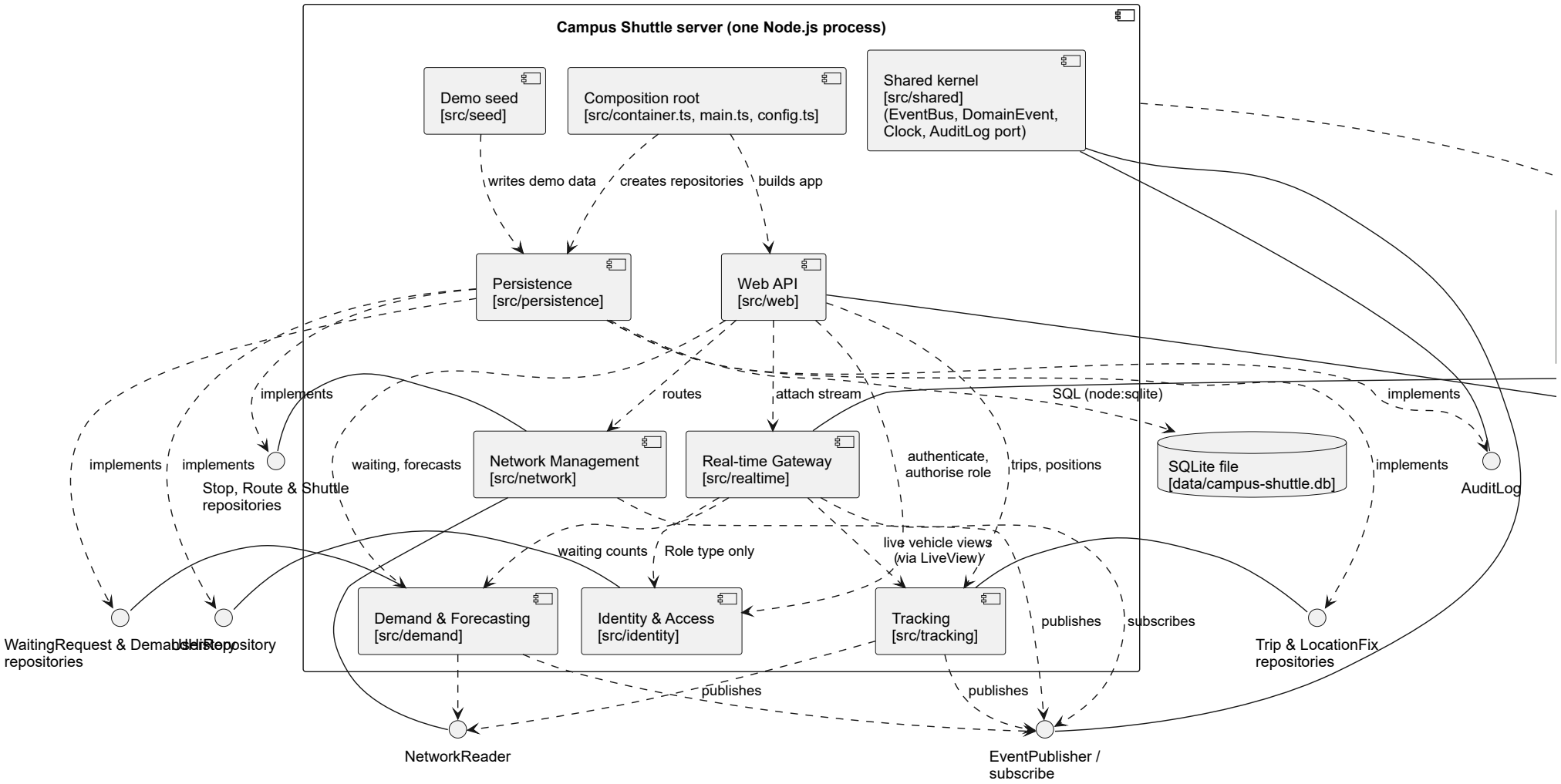


Figure A.11 — Component diagram — detail 1 of 2 (column 1 of 2), at 71 %

ıodular monolith (as implemented; folders in brackets)

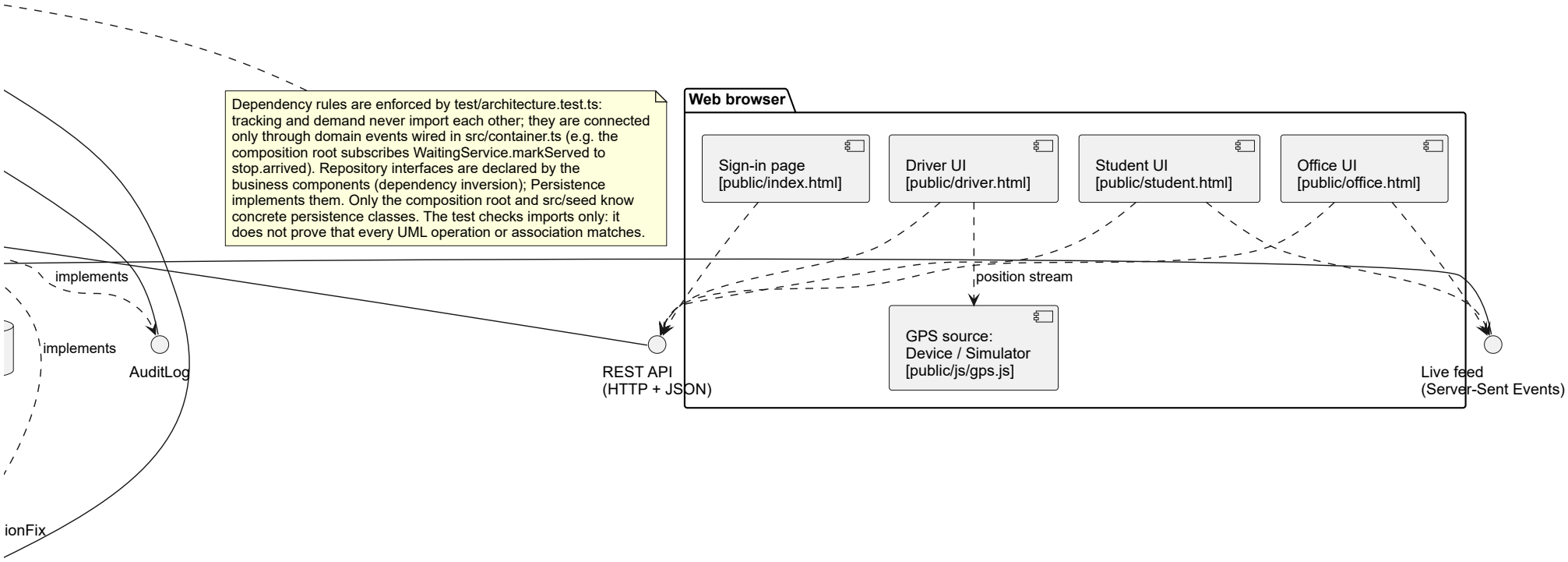


Figure A.11 — Component diagram — detail 2 of 2 (column 2 of 2), at 71 %

A.12 Deployment diagram

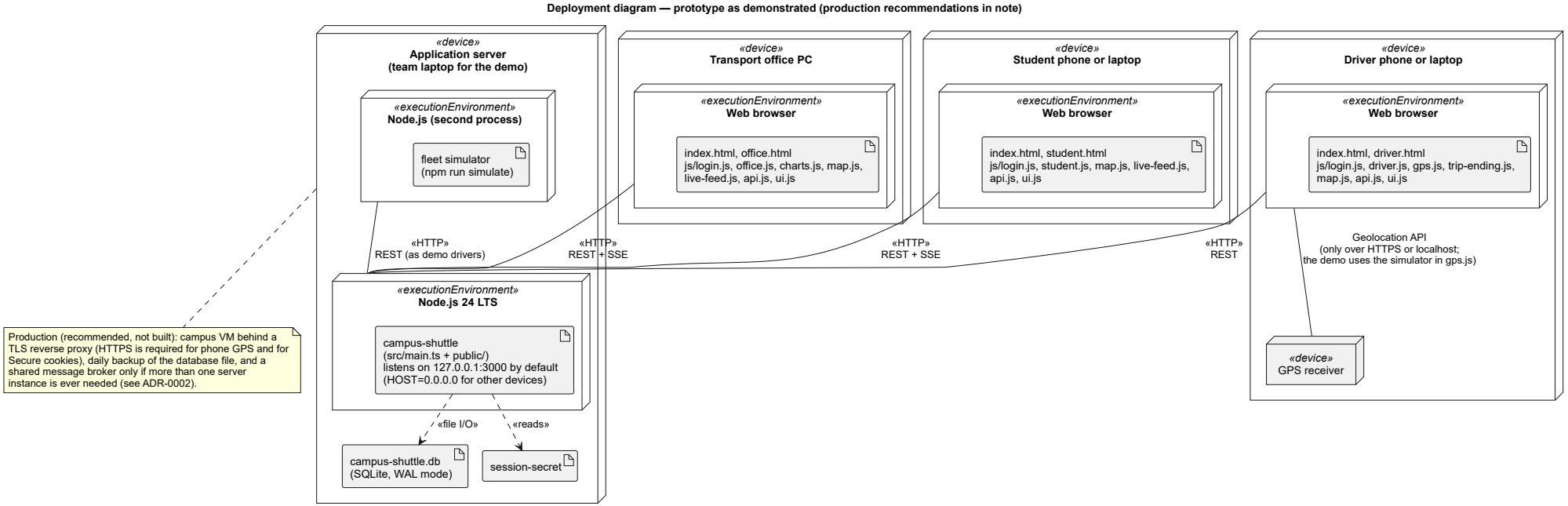


Figure A.12 — Deployment diagram — overview at 51 % (read the 2 detail pages that follow, at 93 %; on screen, zoom in)

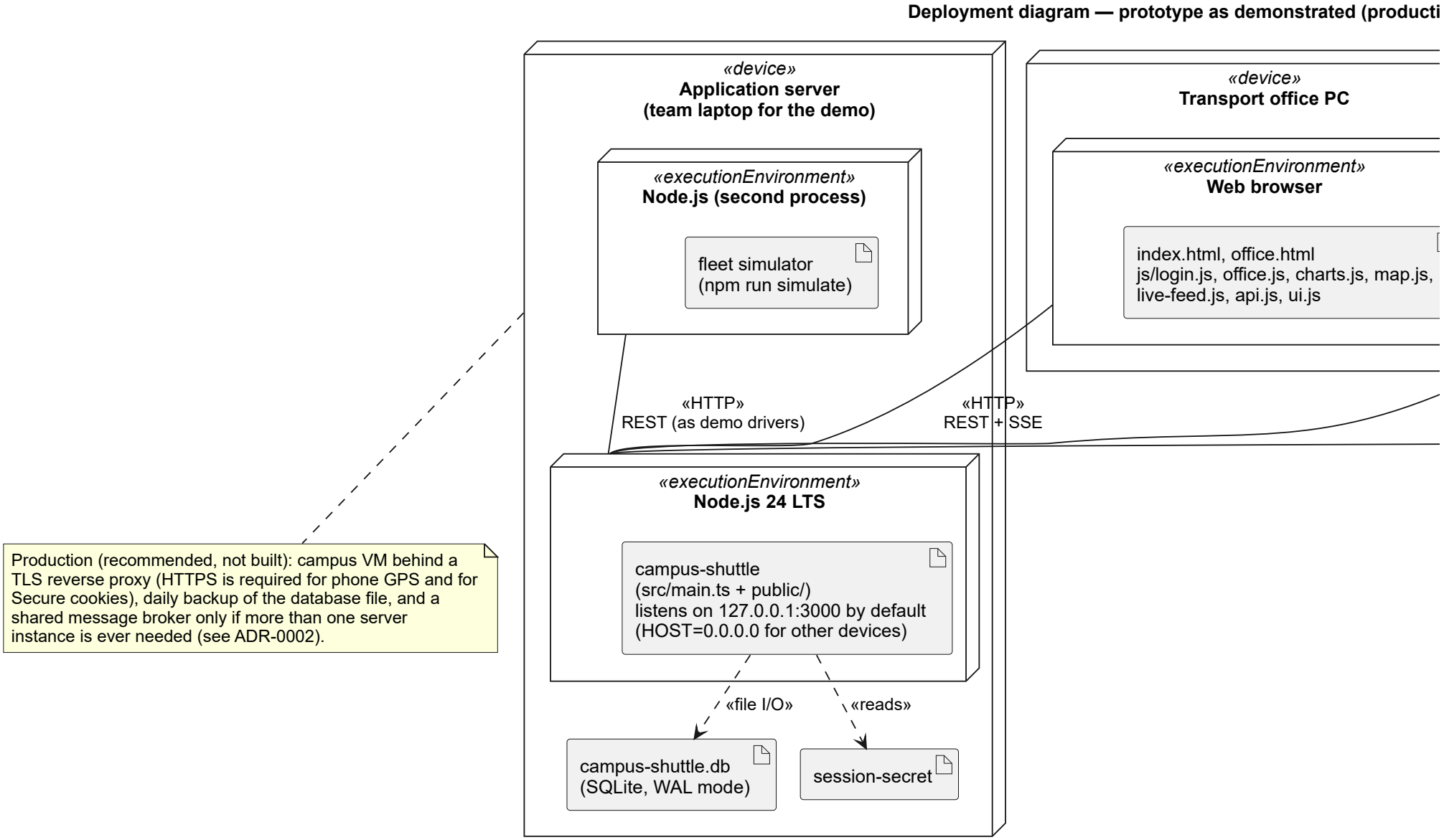


Figure A.12 — Deployment diagram — detail 1 of 2 (column 1 of 2), at 93 %

ype as demonstrated (production recommendations in note)

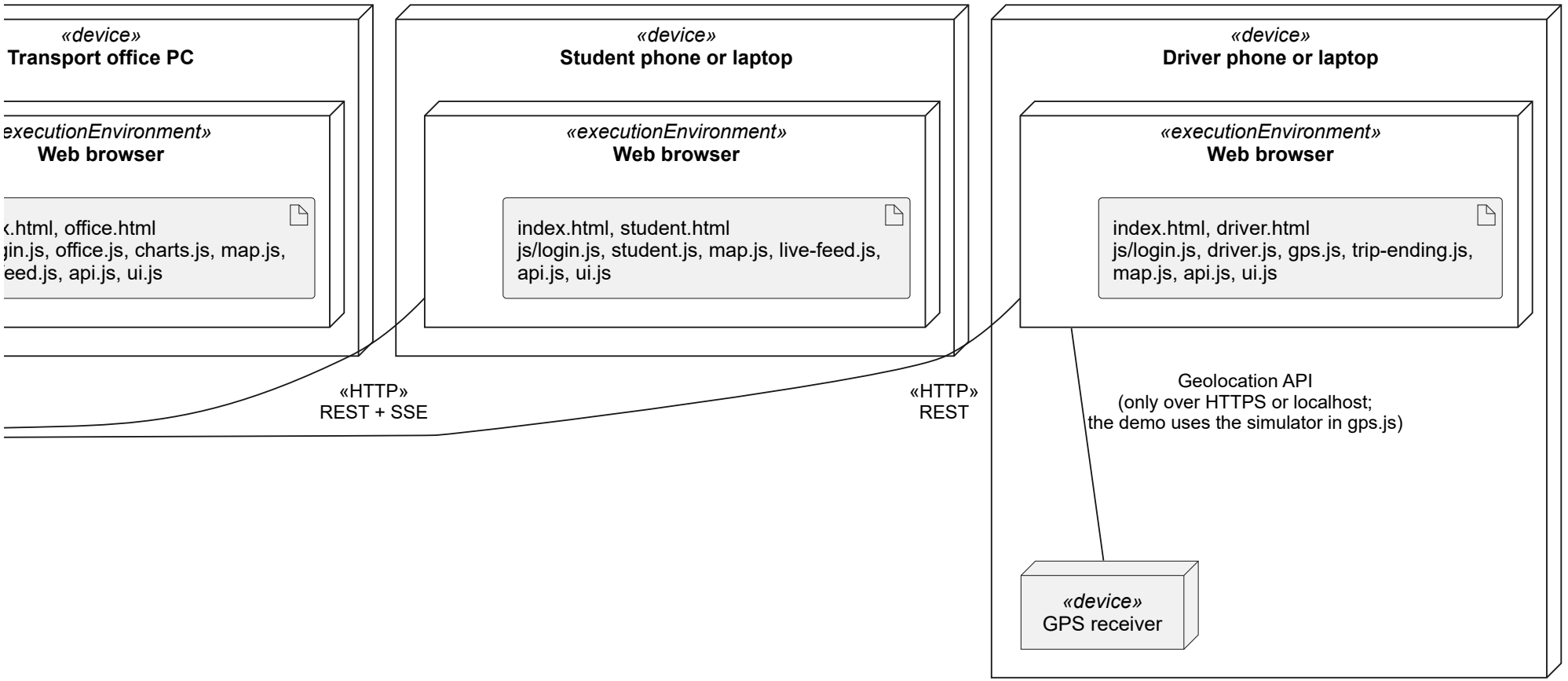


Figure A.12 — Deployment diagram — detail 2 of 2 (column 2 of 2), at 93 %

A.13–A.17 Design pattern class diagrams

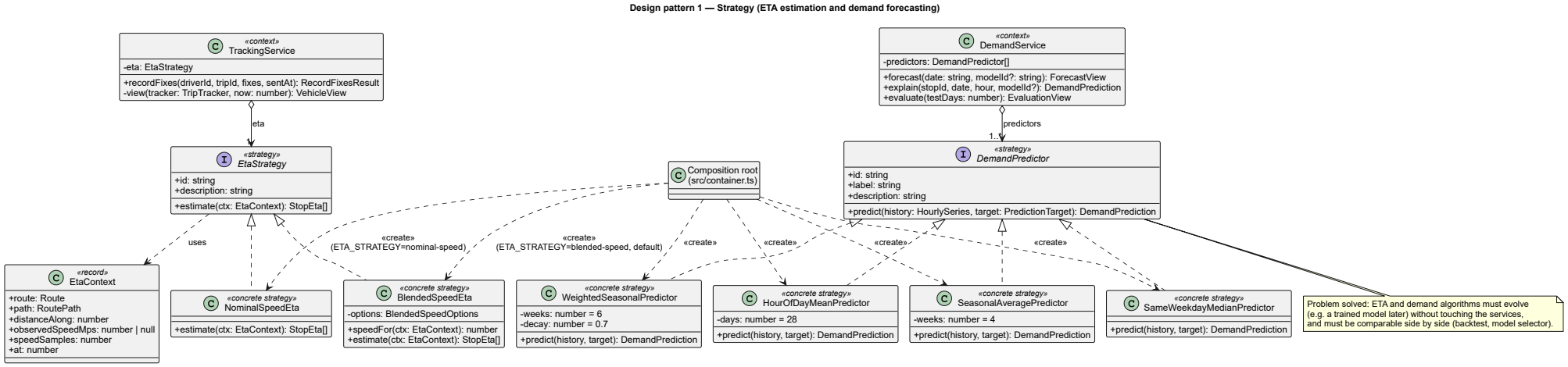


Figure A.13 — Strategy pattern — overview at 43 % (read the 2 detail pages that follow, at 79 %; on screen, zoom in)

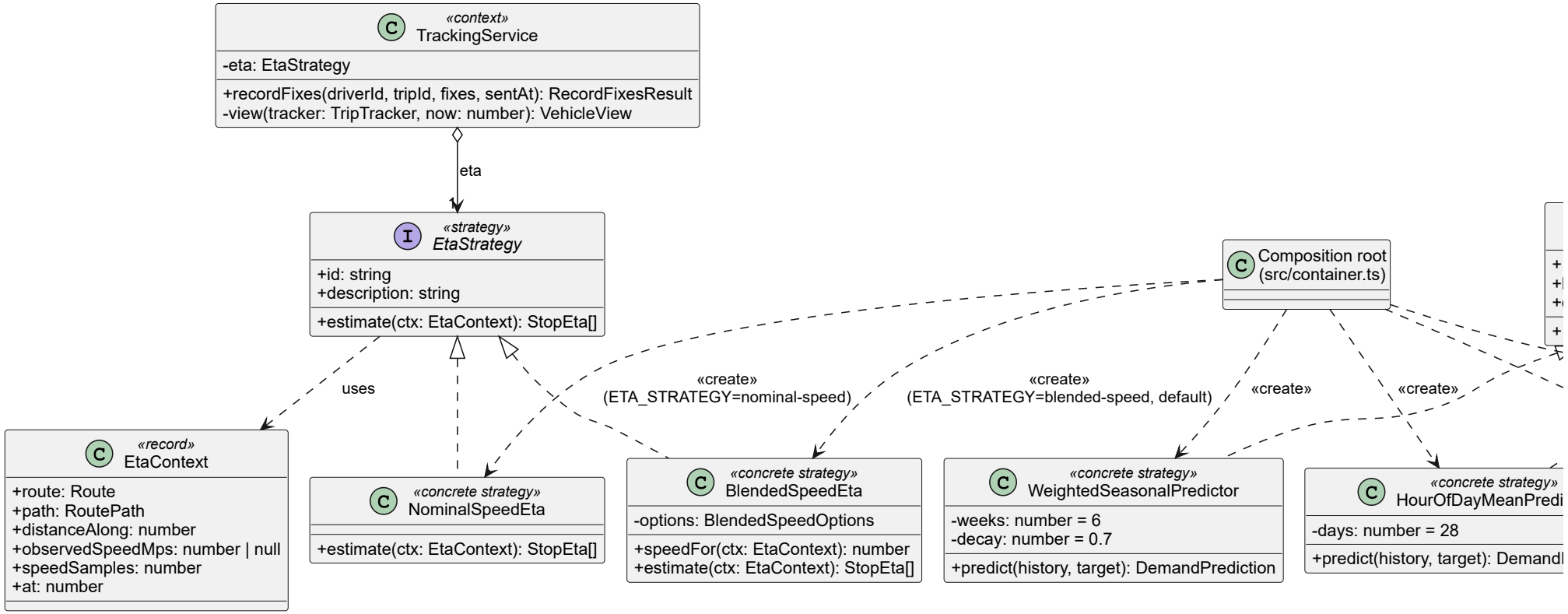


Figure A.13 — Strategy pattern — detail 1 of 2 (column 1 of 2), at 79 %

Strategy (ETA estimation and demand forecasting)

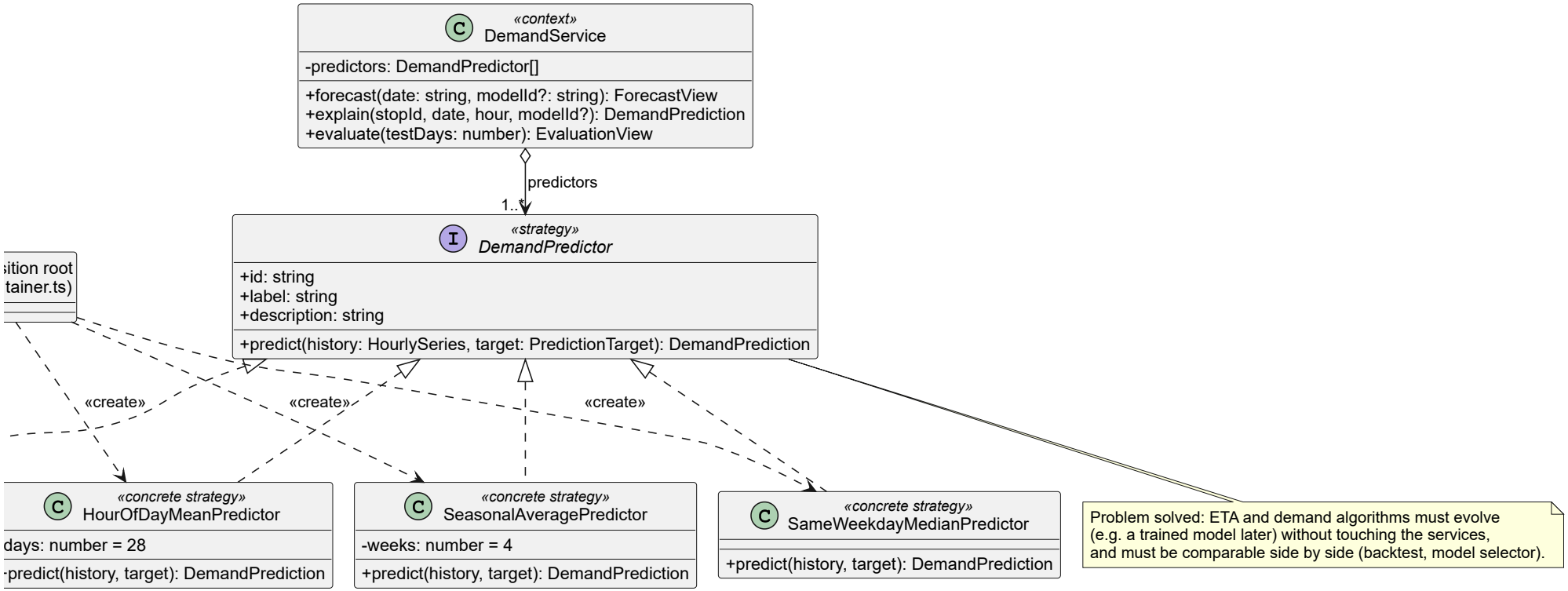


Figure A.13 — Strategy pattern — detail 2 of 2 (column 2 of 2), at 79 %

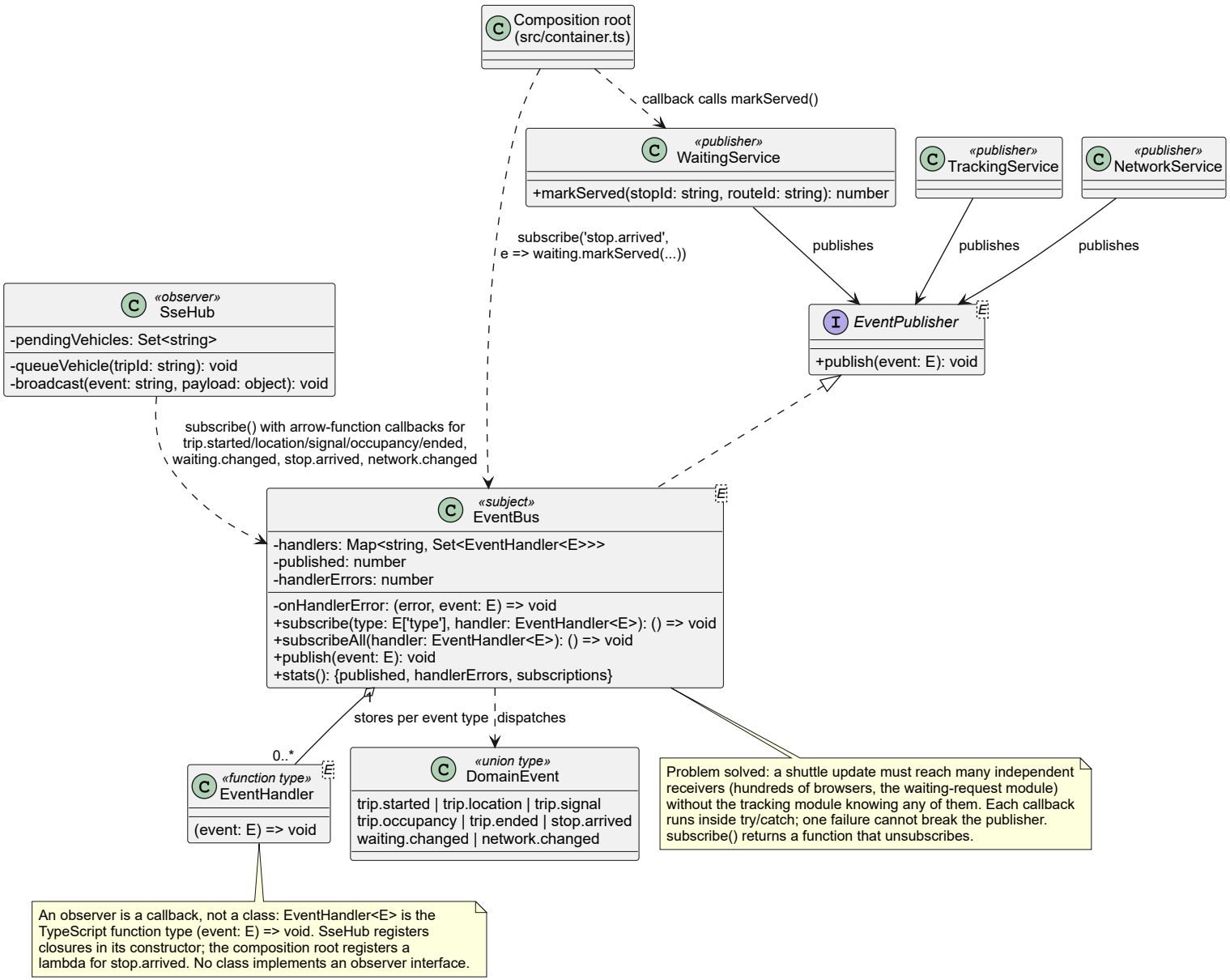


Figure A.14 — Observer pattern

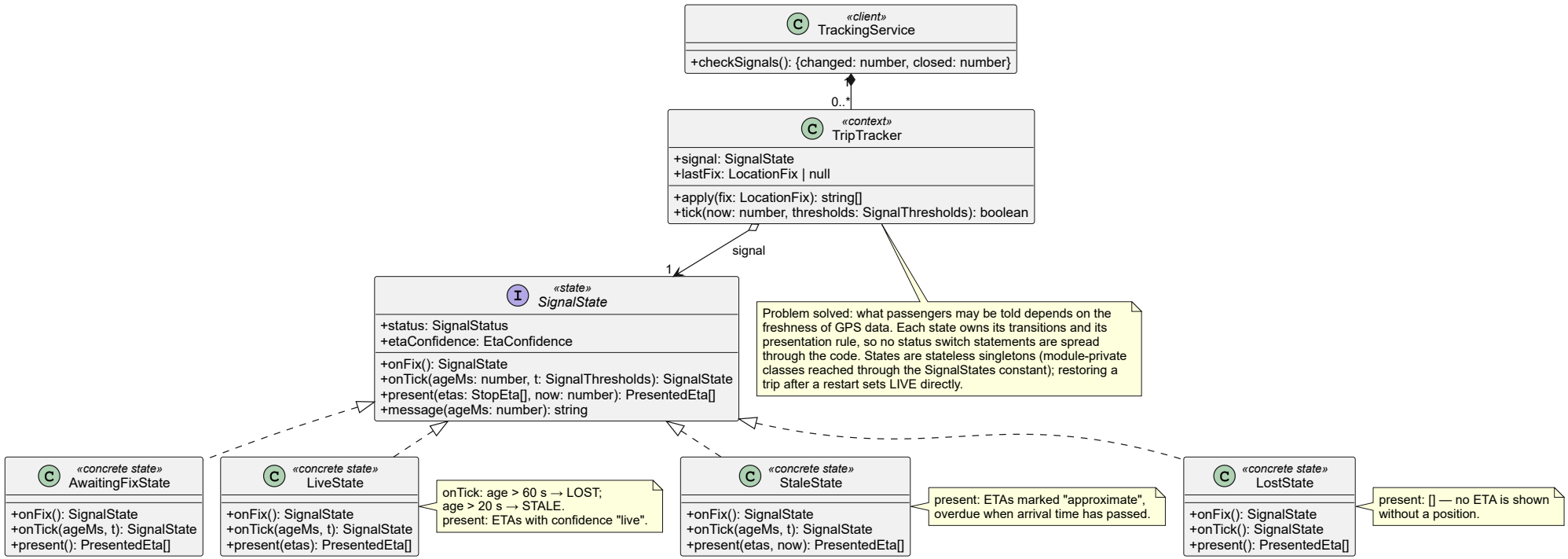


Figure A.15 — State pattern

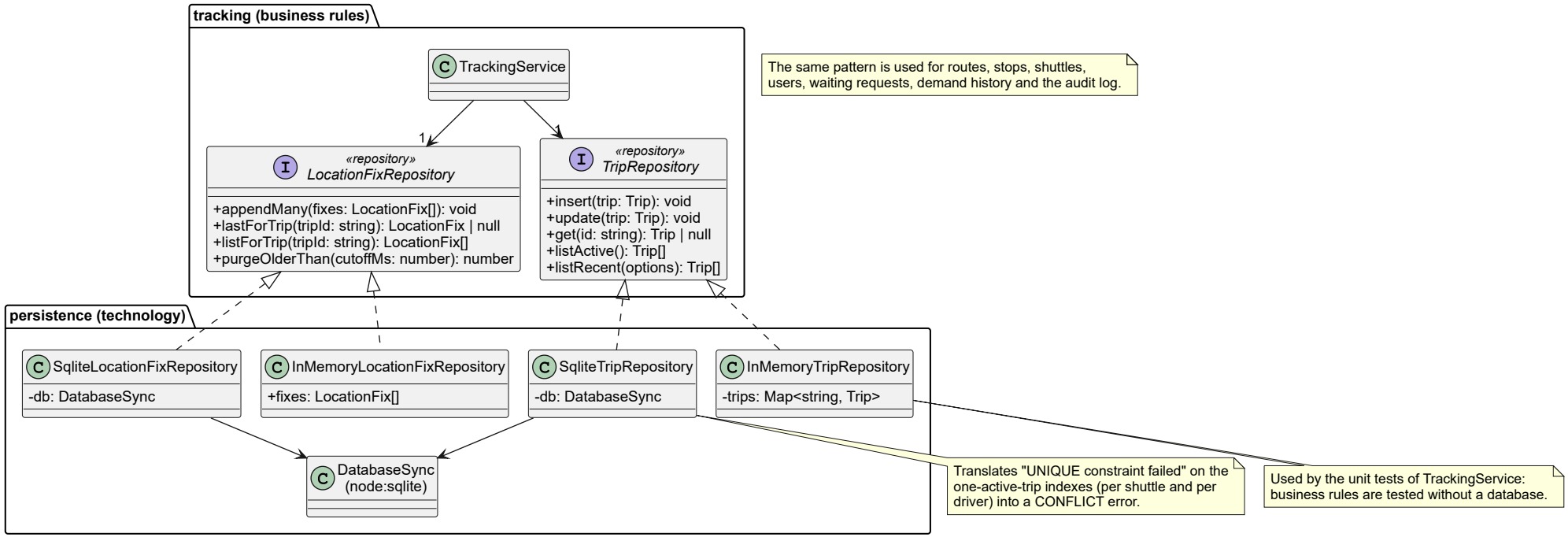


Figure A.16 — Repository pattern

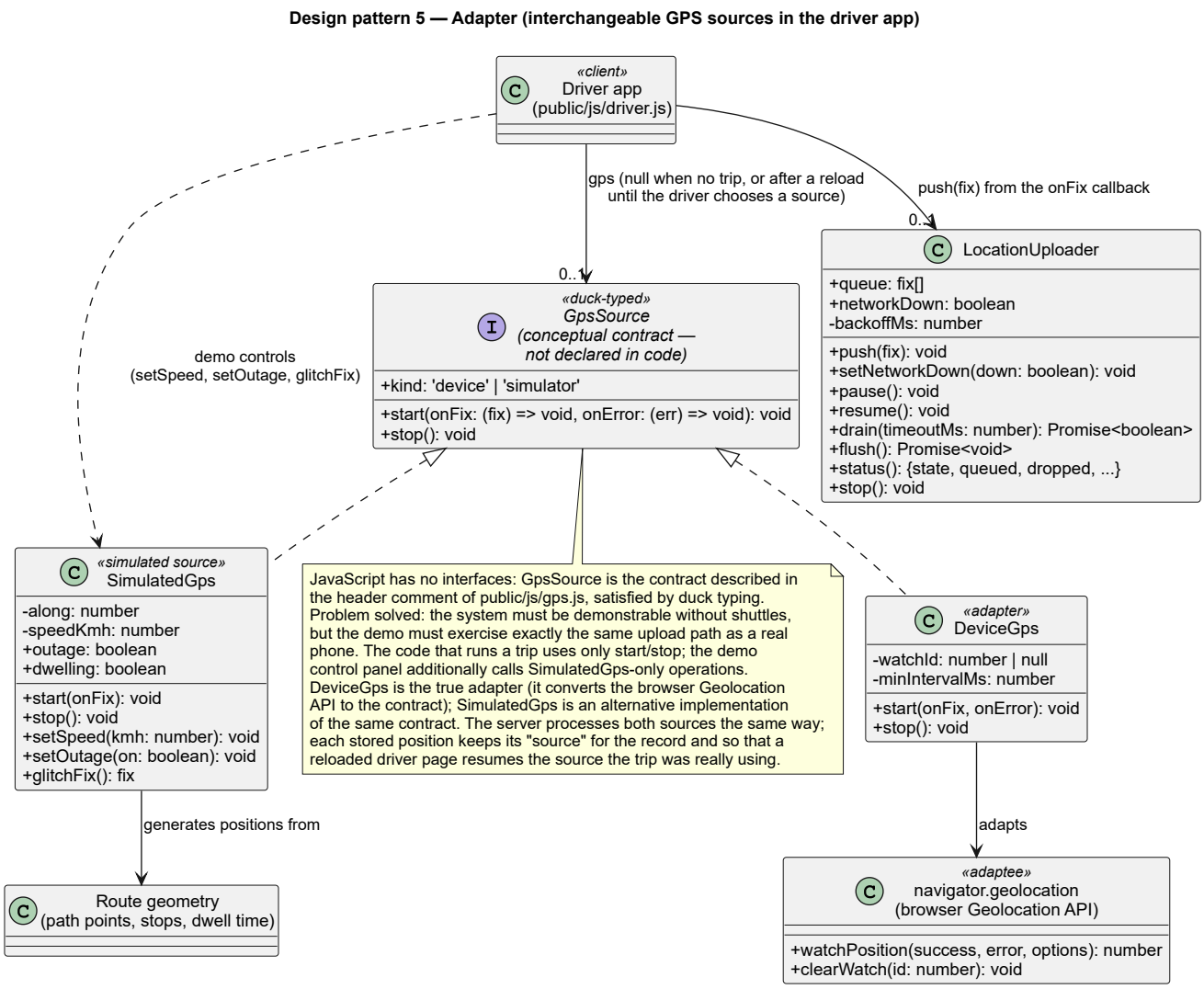


Figure A.17 — Adapter pattern